

Foundations of Computer Networks

NECKER

August 27, 2025

Index

1	Networks and Data Transmission	6
1.1	Buses and Buffers	6
1.2	I/O Ports and Clocks	6
1.3	Calculation of Transmission Time	6
2	Nyquist-Shannon Theorem	6
3	TCP Protocol	6
3.1	Congestion Management	6
3.2	Issues in Wireless Networks	6
4	ACK in TCP	6
4.1	Cumulative ACKs	6
4.2	Duplicate ACKs	7
4.3	Handling of Duplicates	7
5	The OSI Model	7
5.1	The Seven Layers of the OSI Model	7
5.2	Functional Division	7
5.3	Packaging (Encapsulation)	7
5.4	Detailed Example	7
6	Layer Architecture and Functioning	9
6.1	The Data Link Layer (Layer 2)	9
6.1.1	Communication with Adjacent Layers	9
6.1.2	Framing and Delimitation	9
6.2	Bit Stuffing	9
6.2.1	Operation	9
6.2.2	Clarification Point	9
7	Reliability and Channel Utilization	10
7.1	Sequencing of Frames and ACKs	10
7.2	Transmission Efficiency and Channel Utilization	10
7.3	Transmission Efficiency and Channel Utilization	10
7.4	Analysis of Practical Examples	10
8	Window Protocols: Go-Back-N and Selective Repeat	11
8.1	Go-Back-N	11
8.2	Selective Repeat	11
8.3	Comparison: NAK vs. Duplicate ACKs	12
8.4	Confirmation and Retransmission Mechanisms at Layers 2 and 4	12
9	Retransmission Management with Cyclic Numbering	12
9.1	Go-Back-N Protocol and the $k < 2^m - 1$ Rule	12
9.2	Selective Repeat Protocol and the $k \leq 2^{m-1}$ Rule	13

10	Topologies and Channel Access in Local Area Networks (LANs)	13
10.1	LAN Topologies: Broadcast vs. Point-to-Point	13
10.2	Problems and Solutions in Transmission over Broadcast Networks	13
10.2.1	Deterministic Access: Token Ring	14
10.3	CSMA Access Schemes and Persistent Variants	14
10.3.1	Non-Deterministic Access: CSMA/CD in Ethernet	14
10.4	The Binary Exponential Backoff (BEB) Mechanism	14
11	Manchester Encoding	15
12	Efficiency and Ethernet Network Topologies	15
12.1	Channel Efficiency in an Ethernet Network	15
12.2	The Evolution of Topologies: From Hubs to Switches	16
12.2.1	The Concept of a Collision Domain	16
12.2.2	Solution: The Introduction of the Bridge	16
12.2.3	Network Optimization: The Introduction of the Switch	16
12.3	Hybrid Configurations	16
13	Optimization in High-Speed Ethernet Networks	17
13.1	The Latency Problem and Frame Size	17
13.1.1	Calculation of Propagation Time	17
13.2	The Role of the Data Link Layer (Layer 2)	17
14	Virtual LAN (VLAN)	17
14.1	Advantages of VLANs	17
14.2	Operation and Frame Format	18
14.3	Inter-VLAN Communication	18
15	Structure of an IPv4 Packet	19
15.1	Fields of the IPv4 Header	19
15.2	Fragmentation Management	19
15.3	Fragmentation Example	19
15.3.1	Passage over a Token Ring Network (MTU = 4000 B)	20
15.3.2	Passage over an Ethernet Network (MTU = 1500 B)	20
16	IP Addressing Techniques	20
16.1	IP Address Classes	20
16.1.1	Examples of Division	21
16.2	Subnetting	21
16.2.1	Practical Example	21
17	CIDR (Classless Inter-Domain Routing)	22
17.1	Structure and CIDR Allocation	22
17.2	Examples of CIDR Allocation and Calculation	22
17.2.1	Example 1: Calculating the CIDR Mask	22
17.2.2	Example 2: First and Last Available Address	22
17.2.3	Example 3: Subnet Membership	22
18	NAT (Network Address Translation)	22
18.1	What is NAT and what is it used for?	22
18.2	Structure of NAT	23
18.3	How NAT Works	23
18.4	The Problem of Response Management	23
18.5	Violation of the OSI Model: NAT and Layer Manipulation	23
18.6	NAT Operations on Each Packet	24
18.7	NAT and Externally Accessible Servers	24

19 ARP (Address Resolution Protocol)	24
19.1 How Does It Resolve an IP to a MAC Address?	24
19.2 Detailed Functioning of ARP	24
19.3 ARP Packet Format	25
19.4 ARP Across Different Networks: The Role of the Router (Gateway)	25
19.4.1 How is the Problem Resolved?	25
19.4.2 Proxy ARP	25
20 DHCP, ICMP, and Routing	26
20.1 IP Renewal and Release	26
20.2 ICMP Format	26
20.3 Routing vs. Forwarding vs. ARP	26
21 Distance-Vector Routing Protocols	26
21.1 Principles of Operation	26
21.2 RIP (Routing Information Protocol)	26
21.3 Issues with Distance-Vector Protocols	26
21.3.1 Bouncing Effect	27
21.3.2 Count to Infinity	27
21.4 Solutions and Fixes	27
21.4.1 Limitations of Solutions: Why Triggered Updates and Split Horizon Can Fail	27
22 Limits and Applications	27
22.1 Inapplicability on a Large Scale	27
23 Link-State Routing Protocols	28
23.1 Principles of Operation (Link State)	28
23.2 Advantages and Disadvantages of Link-State Protocols	28
23.3 OSPF (Open Shortest Path First)	28
23.3.1 Structure of an OSPF Network: Areas	28
23.3.2 Optimizations in OSPF: Designated Router (DR)	28
24 Large-Scale Routing and Modern Architectures	29
24.1 Routing Between Autonomous Systems: BGP	29
24.2 Evolution: Software Defined Networking (SDN)	29
24.3 Transport Techniques: Tunneling	29
25 Border Gateway Protocol (BGP)	30
25.1 Path Vector Routing and Bouncing Effect Management	30
26 Problems with Traditional Routing	30
27 MPLS: Multi-Protocol Label Switching	30
28 How MPLS Works	30
29 Label Management	31
30 Constructing Switching Tables	31
31 MPLS Packet Format	31
32 Spanning Tree	31
33 Quality Of Service (QoS) in Communication Networks	33
33.1 Queue Management: Input Queuing	33
33.1.1 Random Early Detection (RED)	33
33.1.2 ICMP Choke Packets	33
33.2 Queue Management: Output Queuing	33
33.2.1 The Starvation Problem	33
33.2.2 Weighted Fair Queuing (WFQ) as a Solution	34
33.2.3 Weighted RED (WRED)	34
33.3 Traffic Shaping	34

33.3.1	Token Bucket	34
33.3.2	Call Admission Control (CAC)	34
33.4	Router Protocol Structure	34
34	IPv6: Features and Transition from IPv4	35
34.1	Main Features and Differences with IPv4	35
34.1.1	Address Format and Hierarchical Structure	35
34.2	Compatibility and Transition Mechanisms	35
34.2.1	Dual Stack	35
34.2.2	Tunneling	35
34.2.3	Translation (NAT64)	35
35	TCP Protocol (Transmission Control Protocol)	36
35.1	Managing Multiple Connections on a Server	36
35.2	Connection Process: Socket Primitives	36
35.2.1	Client Side	36
35.2.2	Server Side	36
35.2.3	Data Exchange	36
35.3	TCP Header	36
35.3.1	Pseudo-Header and Checksum	37
35.4	Opening Phase: Three-Way Handshake	37
35.5	Flow Control and Channel Utilization	37
35.5.1	Stall Management: Persist Timer	37
35.6	Idle Connection Management: Keep-Alive Timer	37
36	Congestion Control in TCP	38
36.1	Retransmission Mechanisms: RTO vs. Fast Retransmit	38
36.2	Flow Optimization: Nagle's and Clark's Algorithms	38
36.2.1	To avoid clogging the bandwidth for every individual character, these 2 methods are used: . . .	38
36.3	Congestion Control (Congestion Window - <code>cwnd</code>)	38
36.3.1	Window Growth Phases	38
36.3.2	Reaction to Packet Loss (Congestion)	39
36.3.3	Self-Clocking vs. <code>cwnd</code>	39
37	Advanced Mechanisms in TCP	41
37.1	Dynamic Calculation of Timeout (RTO)	41
37.2	Karn's Rule: Handling Ambiguity in Retransmissions	41
37.3	Improving Reliability: Timestamps and SACK	41
37.4	Connection Teardown (Four-Way Handshake)	41
38	UDP (User Datagram Protocol)	42
38.1	Main Features	42
38.2	When to Use UDP?	42
39	Real-Time Protocols: RTP and RTCP	43
39.1	General Features	43
39.2	RTP (Real-time Transport Protocol) - The Data Flow	43
39.3	RTCP (RTP Control Protocol) - The Control Flow	43
40	DNS (Domain Name System)	44
40.1	Hierarchical Structure and Name Resolution	44
40.2	DNS Records	44
41	Email Architecture	45
41.1	Key Components	45
41.2	Email Sending and Receiving Flow	45
41.3	Protocols for Mail Access	45
41.4	Attachment Management: MIME and Base64	45
41.5	SMTP Structure	45

42 FTP (File Transfer Protocol)	46
42.1 Dual-Connection Architecture	46
42.2 Data Connection Modes: Active vs. Passive	46
42.3 Separation Between Control Plane and Data Plane	46
43 IP Allocation Phases in DHCP	47
44 HTTP (HyperText Transfer Protocol)	48
44.1 HTTP/1.0: Non-Persistent Connections	48
44.2 HTTP/1.1: Persistent Connections and Pipelining	48
44.3 HTTP/2: Multiplexing over TCP	48
44.4 HTTP/3: Shifting to QUIC over UDP	48
45 Difference Between HTTP and HTTPS	49

1 Networks and Data Transmission

1.1 Buses and Buffers

Data moves inside the computer in **parallel** through **buses**, communication channels that connect the various components. Bits are temporarily stored in a memory area called a **buffer** before being sent.

1.2 I/O Ports and Clocks

The **I/O port** converts data from parallel format to **serial** format (one bit at a time) for transmission over network channels. This port is **bidirectional**, allowing data exchange in both directions. The transmission rhythm is paced by a **clock**, a synchronization signal that determines the speed (bps) at which bits are sent.

1.3 Calculation of Transmission Time

The time required to transmit a block of bits is calculated by dividing the number of bits by the transmission speed.

$$T_x = \frac{\text{Number of bits}}{\text{Transmission speed}}$$

Example: To transmit 1000 bits at 1 Mbps (1 million bits per second):

$$T_x = \frac{1000}{10^6} = 10^{-3} \text{ s} = 1 \text{ ms}$$

2 Nyquist-Shannon Theorem

This theorem establishes the requirements for digitizing an analog signal without losing information. The fundamental principle is that the **sampling frequency** (f_s) must be at least twice the **maximum frequency** (f_{max}) present in the original signal.

$$f_s \geq 2 \cdot f_{max}$$

Example: The human voice has an f_{max} of about 4000 Hz. Consequently, the minimum required sampling frequency is 8000 Hz (8 kHz).

3 TCP Protocol

Sends blocks of packets rather than individual packets.

3.1 Congestion Management

TCP is designed to guarantee reliable transmission and manage network **congestion** (routers have buffers; if they are full, incoming packets are discarded). Its strategy consists of slowing down the sending of data when it detects packet loss, assuming that this is due to excessive traffic (since TCP was invented when only wired networks existed, packet loss, being rare, often indicated buffer overflow and automatically triggered a slowdown of the data flow sent by the sender).

3.2 Issues in Wireless Networks

Wireless networks are subject to channel errors (interference, obstacles). TCP can mistakenly interpret packet loss as a sign of congestion, unnecessarily reducing the transmission speed and degrading network performance.

4 ACK in TCP

4.1 Cumulative ACKs

Cumulative ACKs are used to confirm the receipt of all packets in order, up to a certain point in the sequence. For example, if the receiver receives: 1, 2, 3, 5, 6, it sends an ACK for: 1, 2, 3. If after 6 it receives 4, it then sends an ACK for 6 (thus no ACK for 4 or 5 specifically, since it has all of them up to 6).

4.2 Duplicate ACKs

Duplicate ACKs are used to signal the loss of a packet. They are sent when a packet arrives out of sequence, and upon receiving a certain number of these, the sender immediately retransmits the missing packet. For example, if 1, 2, 3, 5, 6 are received, for 5 and 6 I send a duplicate ACK for 4.

4.3 Handling of Duplicates

If, after a retransmission, the receiver receives an identical packet, it **silently discards it**.

5 The OSI Model

- The OSI (Open Systems Interconnection) model is a conceptual subdivision of network functions into seven layers. The objective is to separate the complexity of network communication into simpler parts, facilitating standardization and troubleshooting.

5.1 The Seven Layers of the OSI Model

1. **Layer 1 - Physical:** Handles the transmission of bits at the hardware level (cables, radio waves, etc.). Manages the physical signal.
2. **Layer 2 - Data Link:** Manages the reliable transfer of data between two **adjacent** nodes on the same local area network (LAN) (not just point-to-point).
3. **Layer 3 - Network:** Handles the routing (*routing*) of packets across different networks. **IP** (Internet Protocol) and the router operate at this layer.
4. **Layer 4 - Transport:** Guarantees reliable data transfer between endpoints, managing data segmentation and reassembly. The key protocols are **TCP** and **UDP**.
5. **Layer 5 - Session:** Manages the opening, maintenance, and closing of **communication sessions** between applications.
6. **Layer 6 - Presentation:** Handles data formatting, compression, and encryption, ensuring that information is readable by different systems.
7. **Layer 7 - Application:** Represents the layer closest to the user. It includes the protocols that applications use to access network services (e.g., HTTP, FTP, SMTP).

5.2 Functional Division

The seven layers can be grouped into two macro-categories:

- **Lower Layers (1-4):** They deal with **data transport**. They manage the physical and logical mechanisms to move data from one point to another in the network.
- **Upper Layers (5-7):** They deal with **data processing for applications**. They work closely with the software to ensure data is properly formatted and managed for the end user.

5.3 Packaging (Encapsulation)

When a packet is sent, it passes through all layers, and each layer encapsulates it in a sort of box (adding a header). Once it reaches its destination, the packet is unpackaged through the reverse process.

5.4 Detailed Example

Suppose you want to send the message "Hello" from your computer (IP: 192.168.1.1, MAC: A) to a web server (IP: 203.0.113.10, MAC: B).

Layer 7 - Application: The application (e.g., a web browser) creates the data: "Hello".

Layer 4 - Transport: Adds the TCP header with port numbers (e.g., 80 for the server). The data becomes: [TCP Header]Hello.

Layer 3 - Network: Adds the IP header with the source and destination IP addresses. The data becomes: [IP Header (192.168.1.1 → 203.0.113.10)][TCP Header]Hello.

Layer 2 - Data Link: Adds the MAC header and trailer, specifying the source and destination MAC addresses (which are those of the next router). The data becomes: [MAC Header (A → Router)][IP Header]...[Trailer].

Layer 1 - Physical: The frame is converted into electrical signals and transmitted.

When the packet arrives at the router, it is **decapsulated** up to Layer 3, where the router reads the destination IP address and decides the next path. At that point, the packet is **encapsulated again** with a new MAC header for the next router along the path. This process repeats until the packet reaches the final server, where it is completely decapsulated up to Layer 7.

6 Layer Architecture and Functioning

6.1 The Data Link Layer (Layer 2)

The Data Link Layer handles the reliable transfer of data between two **adjacent** nodes on the same local network. Its scope of action is limited, and it does not have a global view of the network (a data-link layer exists for each node of the router, practically for each possible connection).

6.1.1 Communication with Adjacent Layers

Layer 3 (Network) passes packets to Layer 2, which can manage transmission in two modes:

- **Reliable:** Sends data and waits for an acknowledgment of receipt (a sort of TCP).
- **Unreliable (Best Effort):** Sends data without a guarantee of receipt. This approach is often preferred so as not to slow down the network with continuous confirmations on each individual "hop" between nodes (a sort of UDP), used in ethernet ports.

6.1.2 Framing and Delimitation

To distinguish data frames from the channel in an idle state (idle, a continuous sequence of '1's), the Data Link Layer uses **framing**. It adds a special sequence of bits, called a **flag**, at the beginning and end of each frame.

- The flag is the fixed bit sequence 01111110.
- The sole purpose of the flag is to delimit the beginning and end of data transmission.

WARNING: a frame is not a block of packets.

6.2 Bit Stuffing

Bit stuffing is a technique used by the Data Link Layer to prevent the flag sequence (01111110) from accidentally appearing within the useful data, where it could be misinterpreted as a delimiter.

6.2.1 Operation

1. **Transmitter Side:** The transmitter scans the data to be sent. If it encounters a sequence of **five consecutive '1's**, it automatically inserts a '0' bit right after the fifth '1'.
2. **Receiver Side:** The receiver performs the reverse process. When it receives a sequence of **five consecutive '1's**, it checks the next bit.
 - If the next bit is '0', it considers it a stuffing bit and **discards it**.
 - If the next bit is '1', it recognizes the sequence as a flag (01111110) and concludes frame reception.

6.2.2 Clarification Point

The stuffing bit is removed by the receiver without the original data being altered. The logic is based on the certainty that after five consecutive '1's, any '0' is a stuffing bit added by the sender, unless it is the end-of-frame flag.

7 Reliability and Channel Utilization

7.1 Sequencing of Frames and ACKs

The reliability of a communication protocol requires mechanisms that prevent data loss or duplication. The solution to this problem is the use of **sequence numbers** for both frames and their relative ACKs.

- **Frame Numbering:** Each frame sent by the sender is identified by a progressive sequence number. The receiver expects to receive the frames in a specific order.
- **Role of ACK Sequence Numbers:** It is crucial that ACKs (receipt confirmations) also contain a sequence number that refers to the received frame. This serves to:
 - **Avoid Ambiguity:** Without a sequence number, an ACK could be interpreted as a confirmation for a previous frame, especially if the ACK of a previous frame arrived late.
 - **Manage Retransmission:** If the sender sends Frame 0 but its ACK is lost, the sender retransmits Frame 0. When the receiver receives it, it recognizes it as a duplicate thanks to the sequence number and discards it. If the ACK did not have a number, the sender would have no way of knowing whether a delayed confirmation refers to a previous transmission or not.

7.2 Transmission Efficiency and Channel Utilization

A protocol must be not only reliable but also efficient. Efficiency is measured by **channel utilization** (U), which indicates how much of the channel's capacity is exploited to transmit useful data.

$$U = \frac{t_x}{T}$$

- t_x (**Transmission time**): The time required to send a frame over the channel.
- T (**Total time**): The time that elapses between the start of the frame transmission and the receipt of its ACK.

$$T = t_x + 2t_p$$

In basic protocols, such as **Stop-and-Wait**, the transmitter sends only one frame at a time and remains idle waiting for the ACK. This method is extremely **inefficient** because most of the time the channel remains empty. Efficiency is mainly influenced by:

- **Transmission Time** (t_x): Depends on the size of the frame and the channel bandwidth.
- **Propagation Time** (t_p): The time taken by the signal to travel the distance between sender and receiver.

7.3 Transmission Efficiency and Channel Utilization

The efficiency of a protocol is measured by **channel utilization** (U), which quantifies how much of the available bandwidth is used to transmit useful data. For a *Stop-and-Wait* protocol, which sends only one frame at a time, the formula is:

$$U = \frac{t_x}{T}$$

In window protocols, such as *Sliding Window* (follows TCP logic), the formula changes to reflect the number of frames (k) that can be sent before waiting for a confirmation.

$$U = k \cdot \frac{t_x}{T}$$

7.4 Analysis of Practical Examples

Let us examine how propagation time influences efficiency.

Scenario 1: Short Channel

- **Bandwidth:** 10 Mbps
- **Distance:** 2 km
- **Packet size:** 1000 bits

Calculations:

- $t_x = \frac{1000}{10^7} = 0.1$ ms
- $t_p = \frac{2 \cdot 10^3}{2 \cdot 10^8} = 0.01$ ms (signal speed over cable: $2 \cdot 10^8$ m/s)
- $T = t_x + 2t_p = 0.1 + 0.02 = 0.12$ ms
- **Channel utilization** (U): $U = \frac{0.1}{0.12} \approx 0.83$ (83% utilization)

In this case, t_x is much greater than t_p , making the protocol quite efficient even with the Stop-and-Wait method.

Scenario 2: Long Channel

- **Bandwidth:** 10 Mbps
- **Distance:** 20 km
- **Packet size:** 1000 bits

Calculations:

- $t_x = \frac{1000}{10^7} = 0.1$ ms
- $t_p = \frac{20 \cdot 10^3}{2 \cdot 10^8} = 0.1$ ms (signal speed over cable: $2 \cdot 10^8$ m/s)
- $T = t_x + 2t_p = 0.1 + 0.2 = 0.3$ ms
- **Channel utilization** (U): $U = \frac{0.1}{0.3} \approx 0.33$ (33% utilization)

In this scenario, the propagation time is significant and substantially reduces efficiency. The transmitter wastes a considerable amount of time idle, waiting for the ACK. To maximize channel utilization, multiple frames must be sent in sequence. Since T is 3 times t_x , the transmitter must send at least **3 frames** during each cycle to keep the channel occupied and bring utilization close to 100%.

8 Window Protocols: Go-Back-N and Selective Repeat

Window protocols, such as *Sliding Window* (followed by TCP), increase channel efficiency by allowing the transmitter to send multiple frames consecutively before receiving a confirmation. Depending on how they handle losses, two main approaches are distinguished.

8.1 Go-Back-N

In this protocol, the transmitter sends a sequence of frames, but the receiver has a buffer of limited size, typically **1**. This means that the receiver can only accept frames that arrive in order.

- **Error Handling:** If a frame (e.g., frame 3) is lost or arrives corrupted, the receiver ignores all subsequent frames it receives (e.g., 4, 5, 6), as they are "out of sequence".
- **Retransmission:** The receiver can signal the loss by sending a **NAK** (Negative Acknowledgment) for the missing frame. The transmitter, upon receiving this notification or due to a timeout, retransmits the missing frame and **all subsequent frames** it had already sent. This strategy is simple to implement but is **inefficient**, as it wastes bandwidth retransmitting frames that the receiver had already received correctly (the NAK is sent only once to avoid clogging the network; if it is lost, the packet timer will automatically trigger the retransmission of the packet).

WARNING: (uses cumulative ACKs and NAKs, but they perform the same function).

8.2 Selective Repeat

This protocol is more efficient. The receiver is equipped with a **larger receive buffer** that allows it to accept and store frames that arrive correctly, even if they are out of sequence (WARNING: NAK is the equivalent of a duplicate ACK for the TCP protocol).

- **Error Handling:** When a frame is lost (e.g., frame 3), the receiver continues to accept and store subsequent frames (e.g., 4, 5, 6).
- **Retransmission:** The transmitter, detecting a timeout for frame 3, or at the request of the receiver (e.g., with a selective NAK), retransmits **only the missing frame**.
- **Advantages:** This approach drastically reduces the number of retransmissions and improves channel efficiency. When all frames up to a certain point have been received, the receiver passes them in a block to the upper layer.

8.3 Comparison: NAK vs. Duplicate ACKs

Loss management mechanisms in window protocols can be handled explicitly (NAK) or implicitly (duplicate ACKs).

- **NAK (Negative Acknowledgment):** This is an explicit control message sent by the receiver to notify that a specific packet has been lost or corrupted. Its purpose is to trigger an immediate and targeted retransmission. It is often associated with Go-Back-N protocols.
- **Duplicate ACKs:** This is a technique used by **TCP**. The protocol does not use explicit NAKs. If a packet is lost, the receiver continues to send ACKs confirming the last packet received **in sequence** correctly. For example, if 1, 2, 4, 5 are received, the receiver will respond with an ACK for 2 when it receives 4 and again an ACK for 2 when it receives 5. When the sender receives a predefined number of these duplicate ACKs (generally 3), it assumes that the next packet (in this case, 3) has been lost and retransmits it.

This approach allows TCP to recover from losses without waiting for the timeout, improving performance.

8.4 Confirmation and Retransmission Mechanisms at Layers 2 and 4

- **Layer 4 (TCP)**
 - **Cumulative ACK:** Confirms all bytes received up to a certain sequence number.
 - **Duplicate ACKs:** Signal to the sender that an out-of-order packet has been received.
 - **SACK (Selective Acknowledgment):** Allows indicating exactly which segments have been correctly received.
 - These mechanisms guarantee end-to-end reliability and congestion management.
- **Layer 2 (Data Link)**
 - ACKs generally confirm a *single frame* or, in protocols like Go-Back-N/Selective Repeat, the *next expected frame*.
 - Not cumulative like TCP, advanced handling of out-of-order packets is limited.
 - **Ethernet:** Best effort, no ACK, no NAK.
 - **Wi-Fi (IEEE 802.11):** Mandatory ACK for each frame; absence of ACK = retransmission; does not use explicit NAK.
 - **WAN / PPP / HDLC:** Both ACK and NAK are possible, allowing immediate retransmissions in case of error.

9 Retransmission Management with Cyclic Numbering

When a window protocol uses cyclic frame numbering, which resets after reaching a maximum value, it is crucial that the receiver can distinguish a retransmitted packet from a new packet with the same sequence number. This ambiguity problem is resolved differently depending on the protocol.

9.1 Go-Back-N Protocol and the $k < 2^m - 1$ Rule

In the Go-Back-N protocol, the receiver has a buffer of limited size and accepts only frames that arrive in order. If it receives an out-of-sequence frame, it silently discards it. This seemingly simple logic is the key to avoiding ambiguity, provided that the size of the transmission window (k) is smaller than the size of the sequence number space.

For a sequence number of m bits (which produces 2^m distinct values), the transmission window (k) must satisfy the condition:

$$k < 2^m$$

Example of Ambiguity: Consider a sequence number space of 3 (values from 0 to 2) and a transmission window of size $k = 3$. The sender can send frames 0, 1, 2.

1. The sender sends Frames 0, 1, 2.
2. The receiver receives Frames 0, 1, 2 and sends the related ACKs. All ACKs are lost; now the receiver expects 0 (the fourth packet).
3. Since ACKs 0, 1, 2 do not arrive, the sender resends 0, 1, 2 (the first 3 packets).
4. The receiver receives them but interprets them as packets 4, 5, 6 and not the previous ones: ERROR.

Example of Resolution: Imagine that the sender sends packets 0, 1, 2 and then the ACK for 0 is lost. The sender retransmits packet 0. The receiver, which expects packet 3, receives packet 0 and discards it, since it is not the packet it is expecting. This behavior prevents any ambiguity.

WARNING: if the window (number of frames sent at a time) were equal to the transmission sequence (frame numbering), ambiguity would occur because an expected packet could be either one of the lost ones or a new one.

9.2 Selective Repeat Protocol and the $k \leq 2^{m-1}$ Rule

Unlike Go-Back-N, in Selective Repeat the receiver has a buffer to store packets that arrive out of sequence. While this feature makes the protocol more efficient, it introduces a potentially much more serious ambiguity problem.

If the size of the transmission window (k) is not sufficiently small, a retransmitted packet with a certain sequence number could be confused with a new packet. To prevent this, the size of the transmission and reception windows must never exceed half of the available sequence space.

For a sequence number of m bits (values from 0 to $2^m - 1$), the transmission window (k) must satisfy the condition:

$$k \leq \frac{2^m}{2} \iff k \leq 2^{m-1}$$

Example of Ambiguity (Incorrect Case): Sequence number of 3 bits (values from 0 to 7, $2^3 = 8$) and a transmission and reception window of size $k = 5$.

1. The sender sends Frames 0, 1, 2, 3, 4.
2. Frames 0, 1 are lost. The receiver receives and buffers frames 2, 3, 4.
3. The sender receives the ACKs for 0 and 2, 3, 4. It slides the window and sends frames 5, 6, 7, and 0.
4. The sender retransmits Frame 1.
5. The receiver, which has frames 2, 3, 4 in the buffer and expects Frame 1, receives it correctly and accepts it.

The problem arises when the sender, due to high latency, retransmits packet 0, which has a sequence number already used previously. The receiver might confuse it with a new packet, having no way to distinguish it from the old one.

Explanation: The $k \leq 2^{m-1}$ rule ensures that a retransmitted packet with a sequence number already used can never appear in the receive window. By the time the sequence number repeats, the receiver has already "forgotten" the old packet with the same sequence number, avoiding confusion.

10 Topologies and Channel Access in Local Area Networks (LANs)

10.1 LAN Topologies: Broadcast vs. Point-to-Point

Local area networks (LANs) differ from mesh (or point-to-point) topologies, where each node is connected directly only to its neighbors. Instead, LANs adopt a **broadcast** topology, where every node is connected simultaneously to all others. When a node transmits a signal, it is received by all nodes on the network.

1. **Star topology with passive hub:** One of the earliest broadcast implementations, utilizing a hub as the center of a star. The hub has no intelligence and simply repeats the signal on all its ports, propagating the broadcast.
2. **Linear bus topology:** an historical alternative, where all stations share a single transmission medium. Early versions of Ethernet used this topology, which conceptually is equivalent to a star topology with a passive hub.

10.2 Problems and Solutions in Transmission over Broadcast Networks

The shared nature of broadcast networks introduces two main problems: the need to address packets and the management of contention for channel access.

- **Targeted Addressing:** Each packet contains the addresses of the source and destination nodes. In this way, nodes receive all packets but process only those intended for them.
- **Contention Management:** To avoid collisions due to simultaneous transmissions, channel access protocols have been developed.

10.2.1 Deterministic Access: Token Ring

The Token Ring protocol uses a deterministic mechanism where a **token** (a special packet) circulates along a ring. Only the station holding the token can transmit. This approach guarantees:

- Fair access and no collisions.

Its disadvantages include long waiting times and the need for a central entity to manage the network and recover the token if lost. Due to these limitations, the Token Ring topology was progressively abandoned.

10.3 CSMA Access Schemes and Persistent Variants

- **Non-persistent CSMA without CD (Carrier Sense Multiple Access)**
 - Before transmitting, the station checks if the channel is free.
 - If free → immediate transmission.
 - If busy → waits for a random period of time before rechecking (does not continuously monitor the channel).
 - *Limitation*: Collisions can still occur because there is no collision detection.
- **0-persistent CSMA**
 - If the channel is free → immediate transmission.
 - If busy → waits for a random time interval before retrying.
 - Reduces the probability of collisions compared to pure CSMA.
- **1-persistent CSMA**
 - If the channel is free → immediate transmission.
 - If busy → continues to monitor the channel until it becomes free, then immediate transmission.
 - Higher probability of collision if multiple stations are waiting at the same instant.
- **p-persistent CSMA** (typical for slotted channels)
 - If the channel is free → transmits with probability p , otherwise waits for the next time slot.
 - If the channel is busy → waits until the next free slot.
 - A compromise between collisions and waiting delay.

10.3.1 Non-Deterministic Access: CSMA/CD in Ethernet

Unlike Token Ring, the CSMA/CD (Carrier Sense Multiple Access with Collision Detection) protocol handles channel access in a non-deterministic and distributed manner, accepting the possibility of collisions.

CSMA/CD is an evolution of the **Aloha** protocol, based on three key concepts:

- **Carrier Sense**: Each station "listens" to the channel before transmitting to check if it is free.
- **Collision Detection**: During transmission, each station monitors the channel to detect collisions, meaning the overlap of its own signal with that of another station. In case of a collision, transmission is immediately aborted so as not to waste bandwidth.
- **Exponential Backoff**: After a collision, the station does not retransmit immediately. It waits a random time that increases exponentially with the number of consecutive collisions suffered.

This approach, despite being non-deterministic, guarantees a more efficient use of the channel compared to Aloha.

10.4 The Binary Exponential Backoff (BEB) Mechanism

BEB is a crucial algorithm in CSMA/CD to manage retransmissions after a collision. Its logic is as follows:

$$\text{delay} = \text{random}(0, 2^i - 1) \times T_{\text{slot}}$$

where:

- i is the number of consecutive collisions suffered by the station.
- T_{slot} is the minimum waiting time, defined by the IEEE 802.3 standard.

Definition of Slot Time (T_{slot}): The Slot Time is the maximum round-trip time of a signal in the network, ensuring that a collision can be detected by all stations. For a classic Ethernet network with a maximum length of 2500 meters and a speed of 10 Mbps, the IEEE 802.3 standard defines it as **512 bit times**, which corresponds to 51.2 microseconds. This value, higher than the theoretical propagation time, includes a safety margin to guarantee correct operation under all conditions.

11 Manchester Encoding

- **Purpose:** Manchester encoding is used to transmit digital data so that the receiver can synchronize its own clock with that of the transmitter. Every transmitted bit features a voltage transition at the center of the bit period, ensuring clock recovery.
- **Bit Representation:**
 - $0 \rightarrow \text{high} \rightarrow \text{low}$ transition at mid-bit.
 - $1 \rightarrow \text{low} \rightarrow \text{high}$ transition at mid-bit.
- **Preamble and Start/End Flags:**
 - At the beginning of an Ethernet transmission, a **preamble** consisting of 7 bytes of alternating 10101010 bits is sent (in Manchester: regular transitions), followed by a 1-byte **Start Frame Delimiter (SFD)** 10101011.
 - The preamble allows the receiver to synchronize its clock with the transmitter.
 - Initial and final flags are eliminated during Manchester encoding because the receiver does not need external signals: the clock is extracted directly from the bit transitions.
- **Handling Consecutive Sequences of 1s or 0s:**
 - If identical consecutive bits are transmitted, for example 1111, Manchester encoding generates transitions at mid-bit for each bit.
 - The initial transition of a new identical bit can be ignored by the receiver as a "setup" and not as a valid transition for the clock. This prevents the continuity of the high or low level from falsifying synchronization.
 - In practice, the receiver measures the time difference between transitions to correctly determine each bit, ignoring setup transitions when necessary.
- **Main Advantage:** Allows the receiver to recover the clock directly from the transmitted data, eliminating the need for a separate channel for synchronization.

This technique doubles the transmission clock speed to ensure that transitions occur exactly at the midpoint of each bit period, allowing the receiver to synchronize with the data stream.

12 Efficiency and Ethernet Network Topologies

12.1 Channel Efficiency in an Ethernet Network

The efficiency of an Ethernet channel depends on the management of **contentions**, meaning the probability that multiple stations attempt to transmit simultaneously, causing **collisions**. The efficiency of channel utilization (U) in a point-to-point network can be expressed as:

$$U = \frac{t_x}{t_x + 2t_p \times e}$$

where:

- t_x is the frame transmission time (L_{frame}/B).
- t_p is the signal propagation time (L_{channel}/v_p).
- e is a parameter that accounts for the impact of collisions.

Substituting the values, the formula becomes:

$$U = \frac{1}{1 + 2 \times \frac{B \times L_{\text{channel}}}{L_{\text{frame}} \times v_p} \times e}$$

This formula highlights that an increase in bandwidth (B) or channel length (L_{channel}) reduces efficiency. This occurs because propagation time has a greater impact than transmission time, increasing the risk and duration of collisions.

12.2 The Evolution of Topologies: From Hubs to Switches

Early Ethernet networks adopted broadcast topologies (like the bus or the star with a hub) where all stations shared the same transmission medium. The entire network constituted a single, large **collision domain**. As the number of stations grew, the probability of collision increased, degrading performance.

12.2.1 The Concept of a Collision Domain

A collision domain is a network segment where a collision between two packets can be detected by all stations. In hub-based networks, a collision anywhere on the cable propagates to all nodes, forcing everyone to stop transmitting and restart the exponential backoff process.

12.2.2 Solution: The Introduction of the Bridge

To solve this problem, the **bridge** was introduced, an intelligent Layer 2 device (Data Link Layer). The bridge acts as a "bridge" between two or more network segments, separating traffic between them and, consequently, dividing a large collision domain into multiple smaller ones.

- **Operation:** Unlike a hub, which blindly repeats signals, the bridge examines the destination MAC address of each frame.
- **Filtering and Forwarding:** If the sender and destination are on the same network segment, the bridge discards the frame. If the destination is on another segment, the bridge forwards the frame only to the correct port.
- **Forwarding Table:** The bridge constructs a table dynamically, associating the MAC addresses of stations with connection ports. If an address is unknown, the bridge performs a *flooding* (forwards the packet to all ports), learning the destination's location as soon as it responds.

12.2.3 Network Optimization: The Introduction of the Switch

The **switch** represents the evolution of the bridge, a Layer 2 device that further optimizes network traffic and virtually eliminates collisions.

- **Point-to-Point Connections:** Unlike a bridge that connects entire segments, a switch creates a separate collision domain for each individual port. This means that each device connected to the switch has a dedicated channel.
- **Elimination of Collisions:** Thanks to these dedicated connections, collisions no longer occur between devices connected to different ports of the switch. This makes the use of the CSMA/CD protocol unnecessary on most of the network, drastically improving performance and throughput.
- **Superior Performance:** Multiple devices can communicate simultaneously without interference, reducing latency and increasing overall efficiency.

12.3 Hybrid Configurations

In modern networks, the most efficient configuration involves using **switches** as central devices, guaranteeing a collision-free network, and potentially deploying **hubs** only locally, to connect groups of devices to a single switch port. This approach limits collision domains to small portions of the network, maintaining scalability and high performance centrally.

13 Optimization in High-Speed Ethernet Networks

The evolution of Ethernet networks has led to a significant increase in transmission speed, requiring the optimization of the access protocol to guarantee efficiency and correct collision management.

13.1 The Latency Problem and Frame Size

Increasing transmission speed reduces the time required to send a frame. To guarantee that a station can detect a collision while it is still transmitting (according to the CSMA/CD protocol), it is necessary for the frame transmission time to be greater than the maximum signal propagation time in the cable.

13.1.1 Calculation of Propagation Time

Considering a star topology with a hub and a maximum cable length of 200 m per segment, the maximum propagation time for a collision (round-trip time) is calculated over a total distance of 800 m (from A to HUB, from HUB to C, from C to HUB, from HUB to A).

$$\text{Propagation time} = \frac{\text{Distance}}{\text{Propagation speed}} = \frac{800 \text{ m}}{2 \times 10^8 \text{ m/s}} = 4 \mu\text{s}$$

At a speed of 1 Gbps, in 4 μs one can transmit:

$$1 \text{ Gbps} \times 4 \mu\text{s} = 10^9 \text{ bps} \times 4 \times 10^{-6} \text{ s} = 4000 \text{ bits} = 500 \text{ bytes}$$

For this reason, the **Gigabit Ethernet** standard modified the minimum frame size from 64 to **512 bytes** and reduced the maximum channel length to 800 m, to guarantee that transmission time exceeds the maximum propagation time, allowing correct collision management.

WARNING: if older devices are used (at 10 Mbps, for example), they would theoretically continue to broadcast only 64 bytes (too short); for this reason, if a Gigabit port exists, it takes care of adding padding to always reach 512 bytes.

13.2 The Role of the Data Link Layer (Layer 2)

The Data Link layer handles communication between devices directly connected on a network and is divided into two sublayers.

- **MAC (Media Access Control)**: Manages access to the transmission medium and the format of frames. In shared-medium networks (like those with hubs), the MAC uses protocols like CSMA/CD. With the use of switches, the MAC manages dedicated channels, making CSMA/CD no longer necessary.
- **LLC (Logical Link Control)**: Provides an abstraction of the MAC layer to upper layers, hiding the complexity of the shared medium and presenting the connection as a simple point-to-point link.

14 Virtual LAN (VLAN)

A VLAN is a logical abstraction of a network that allows grouping devices independently of their physical location.

14.1 Advantages of VLANs

- **Security**: Machines in different VLANs are isolated and cannot communicate with each other at Layer 2, reducing the risk of unauthorized access (this is because physical communication requires an extra parameter, namely the VLAN ID).
- **Flexibility**: Devices can be grouped based on their function, not their physical location.
- **Traffic Optimization**: Broadcast traffic is confined within its own VLAN, reducing congestion and improving network performance.

14.2 Operation and Frame Format

Switches support VLANs through an **extended forwarding table**, which associates not only the MAC address and port, but also a **VLAN ID**.

- **Untagged Frames:** These are frames sent from a host to a switch or vice versa. They do not contain any VLAN information, which is assigned by the switch based on the ingress port, used for communications on the same VLAN (the switch maps the MAC to the VLAN ID to see if they are on the same network).
- **Tagged Frames:** These are frames traveling between switches over a **trunk** link. They are modified with the addition of an **802.1Q tag** containing the VLAN ID (thus in communications between PCs on different VLANs) only if a Layer 3 device exists.

The addition of the 802.1Q tag increases the size of the Ethernet frame.

14.3 Inter-VLAN Communication

To allow communication between different VLANs, a device operating at **Layer 3** is required, such as a **router** or a **Layer 3 switch**. These devices can route packets based on IP addresses, overcoming the Layer 2 isolation enforced by VLANs.

15 Structure of an IPv4 Packet

An IPv4 packet is composed of a **header** and a **payload**. The standard header has a length of 20 bytes but can be extended with options.

15.1 Fields of the IPv4 Header

The IPv4 header is divided into numerous fields that define its operation.

1. **Version (4 bits)**: Indicates the version of the IP protocol (4 for IPv4).
2. **Header Length (IHL) (4 bits)**: Specifies the header length in 32-bit words.
3. **Type of Service (ToS) (8 bits)**: Defines packet priority.
4. **Total Length (16 bits)**: Indicates the total length of the packet (header + payload) in bytes.
5. **Identification (16 bits)**: Identification number of the original packet, used for reassembling fragments.
6. **Flags (3 bits)**:
 - **D bit (Don't Fragment)**: Prevents fragmentation.
 - **M bit (More Fragments)**: Indicates that there are subsequent fragments.
7. **Fragment Offset (13 bits)**: Indicates the position of a fragment within the original packet, in multiples of 8 bytes.
8. **Time To Live (TTL) (8 bits)**: Maximum number of "hops" (routers) the packet can traverse.
9. **Protocol Selector (8 bits)**: Indicates the upper-layer protocol (e.g., TCP or UDP) of the payload.
10. **Header Checksum (16 bits)**: Error control on the header, recalculated at each hop.
11. **Source IP Address (32 bits)**: The sender's IP address.
12. **Destination IP Address (32 bits)**: The recipient's IP address.

(The packet also contains fields for options [an optional field used for advanced features, like source routing or route recording] and the payload [the actual data transported by the packet]).

15.2 Fragmentation Management

Fragmentation occurs at two levels:

1. At **Layer 4** (e.g., TCP): Segmentation of user data into packets with a maximum size of 65,535 bytes.
2. At **Layer 3** (IP): Occurs when a packet exceeds the **Maximum Transmission Unit (MTU)** of the underlying network, adapting packets to the maximum size allowed by the physical layer.

The process relies on header fields (for Layer 3):

- Each fragment receives the same **Identification** as the original packet.
- The **Flags** (specifically the M bit) indicate if more fragments follow.
- The **Fragment Offset** specifies the position of the fragment in the original packet, using units of 8 bytes.

Reassembly of fragments occurs only on the final destination device.

15.3 Fragmentation Example

Consider an original packet of 7000 bytes traversing two networks with different MTUs. The original IP packet has a 20-byte header and a 6980-byte payload.

15.3.1 Passage over a Token Ring Network (MTU = 4000 B)

The original 7000 B packet exceeds the MTU of 4000 B. It is therefore fragmented into two parts, each with a new 20-byte IP header. The payload of each fragment cannot exceed $4000 - 20 = 3980$ B, and must be a multiple of 8.

- **First fragment:**

- Payload size: 3976 B (3976 is a multiple of 8).
- Total size: $3976 + 20$ (header) = 3996 B.
- Fragment Offset: 0 (first fragment).
- More Fragments (MF) bit: Set to 1.

- **Second fragment:**

- Payload size: $7000 - 20$ (original header of the 7000B packet) $- 3976 = 3024$ B.
- Total size: $3024 + 20$ (header) = 3044 B.
- Fragment Offset: $\frac{3976}{8} = 497$.
- More Fragments (MF) bit: Set to 0 (last fragment).

15.3.2 Passage over an Ethernet Network (MTU = 1500 B)

The second router receives the two previous fragments and must fragment them further, as both exceed the MTU of 1500 B. The maximum payload size of a fragment is $1500 - 20$ (header) = 1480 B.

- **Fragmentation of the first fragment (3996 B total / 3976 B payload):**

- **Sub-fragment 1:** payload 1480 B, offset 0.
- **Sub-fragment 2:** payload 1480 B, offset $\frac{1480}{8} = 185$.
- **Sub-fragment 3:** payload $3976 - 1480 - 1480 = 1016$ B, offset $\frac{1480+1480}{8} = 370$.

- **Fragmentation of the second fragment (3044 B total / 3024 B payload):**

- **Sub-fragment 4:** payload 1480 B, offset $370 + \frac{1016}{8} = 370 + 127 = 497$.
- **Sub-fragment 5:** payload 1016 B, offset $497 + \frac{1480}{8} = 497 + 185 = 682$.
- **Sub-fragment 6:** payload 528 B, offset $682 + \frac{1016}{8} = 682 + 127 = 809$.

16 IP Addressing Techniques

IP addressing is managed by ICANN and regional registries. Initially, IPv4 addresses were organized into classes.

16.1 IP Address Classes

IPv4 addressing is divided into classes (A, B, C) based on the value of the first octet. This classification determines the division between the network identifier (**Net ID**) and the host identifier (**Host ID**).

- **Class A** (first octet for net ID, the rest for host): The first bit is **0**. The address is divided into **7 bits for the Net ID** and **24 bits for the Host ID**. It is suitable for very large networks. The range of the first octet is from 1 to 127.
- **Class B** (first and second octets for net ID, the rest for host): The first two bits are **10**. The address is divided into **14 bits for the Net ID** and **16 bits for the Host ID**. It is suitable for medium-sized networks. The range of the first octet is from 128 to 191.
- **Class C** (first, second, and third octets for net ID, the rest for host): The first three bits are **110**. The address is divided into **21 bits for the Net ID** and **8 bits for the Host ID**. It is suitable for small networks. The range of the first octet is from 192 to 223.
- **Class D/E**: (D) Multicast for routers with the same IP, (E) reserved for future or experimental use; neither has a network part and a host part. First octet ranges from 224 to 255.

16.1.1 Examples of Division

The class of an IP address determines which octets identify the network and which identify the host.

- **Class A:** The address 10.255.255.255 is a Class A address. The **Net ID** is 10, and the **Host ID** is 255.255.255. All devices on the 10.0.0.0 network will have the same Net ID, while the rest of the address identifies the individual host.
- **Class B:** An address like 172.16.255.255 belongs to Class B. The **Net ID** is 172.16 and the **Host ID** is 255.255. This address identifies the last possible host within the 172.16.0.0 network.
- **Class C:** The address 192.168.1.255 belongs to Class C. The **Net ID** is 192.168.1 and the **Host ID** is 255.

This approach facilitates routing: a router examines only the Net ID. If the source Net ID and destination Net ID are equal, the packet remains in the local network; if they are different, it is routed toward the egress gateway.

16.2 Subnetting

The class-based approach proved inefficient, causing a waste of addresses. To solve this problem, **Subnetting** was introduced, a technique that allows dividing a large network into smaller subnetworks.

- Subnetting introduces an additional hierarchical layer, using a portion of the Host ID bits to create a subnetwork identifier.
- The **subnet mask** is used to determine a host's subnet. Performing a **bitwise AND** operation between the IP address and the subnet mask yields the subnetwork address.
- **CIDR** notation (e.g., /22) indicates the length of the network and subnet portion of the address. For example, '/22' means that 22 bits (the bits set to 1 in the subnet mask) are used to identify the network and subnet, while the remaining bits (set to 0) identify the hosts.

This technique has optimized the use of the IPv4 address space.

16.2.1 Practical Example

Consider the IP address 172.16.20.50 and a subnet mask 255.255.255.0 (CIDR notation: /24). The subnetwork address is obtained by applying the **bitwise AND** operation between the IP address and the subnet mask.

```
IP Address:   10101100.00010000.00010100.00110010
Subnet Mask:  11111111.11111111.11111111.00000000
AND Result:   10101100.00010000.00010100.00000000
```

The result, converted to dotted decimal notation, is 172.16.20.0, which represents the subnetwork address. All devices with an address starting with 172.16.20. belong to this same subnetwork.

17 CIDR (Classless Inter-Domain Routing)

CIDR (Classless Inter-Domain Routing) was introduced to optimize the use of IPv4 addresses and slow down the depletion of available space. Unlike the previous class-based scheme (A, B, C), CIDR allows allocating address blocks based on actual needs, reducing IP waste and improving routing efficiency.

17.1 Structure and CIDR Allocation

With CIDR, IP addresses are allocated in variable-sized blocks, identified by a special notation: `IP_ADDRESS/SUBNET_MASK`.

Where `SUBNET_MASK` represents the number of bits used to identify the network. For example, `192.168.1.0/24` indicates that the first 24 bits are reserved for the network, leaving the last 8 bits for the hosts (2^8 possible addresses).

17.2 Examples of CIDR Allocation and Calculation

Let's analyze some examples to understand how CIDR optimizes IP address allocation and management.

17.2.1 Example 1: Calculating the CIDR Mask

Question: A company requires 4000 IP addresses. What is the minimum necessary CIDR mask?

Response:

- The number of required hosts is 4000. The nearest power of 2 is 4096 (2^{12}).
- 12 bits are required for the hosts. Since an IPv4 address consists of 32 bits, the CIDR mask is calculated as $32 - 12 = 20$.
- The minimum necessary subnet mask is **/20**.

17.2.2 Example 2: First and Last Available Address

Question: Given the block `10.10.32.0/20`, what are the first and last available addresses?

Response:

- The `/20` mask leaves $32 - 20 = 12$ bits for hosts, so the number of addresses is $2^{12} = 4096$.
- The range is from `10.10.32.0` to `10.10.47.255`.
- The first usable address is `10.10.32.1`.
- The last usable address is `10.10.47.254`.

17.2.3 Example 3: Subnet Membership

Question: An ISP assigns the range `172.16.64.0/22`. Does the address `172.16.66.25` belong to this network?

Response:

- The `/22` mask leaves $32 - 22 = 10$ bits for hosts. The address range goes from `172.16.64.0` to `172.16.67.255`.
- The address `172.16.66.25` falls within this interval, so **yes**, the IP belongs to the subnet.

CIDR extended the lifespan of IPv4, but did not make it eternal, necessitating the adoption of other technologies like NAT for its management.

18 NAT (Network Address Translation)

The adoption of NAT (Network Address Translation) has significantly contributed to extending the lifespan of IPv4, although it has not made it eternal. Many networks continue to function with IPv4 thanks to NAT, which allows optimized management of IP addresses.

18.1 What is NAT and what is it used for?

NAT is a technology that allows translating private IP addresses into a public IP address and vice versa. This mechanism allows optimizing the use of available IPv4 addresses, protecting the private network from direct external access, and separating public address space from private space.

18.2 Structure of NAT

In a private network, private IP addresses can be assigned to devices, which are not visible on the Internet. A NAT device, typically a router, possesses a single public IP address and acts as an intermediary between the private network and the outside world. The private IP addresses usable within an internal network belong to the following reserved ranges:

- 10.0.0.0/8
- 172.16.0.0/12
- 192.168.0.0/16

Note: These addresses can be reused across multiple networks without causing conflicts, as private network traffic is not routable on the Internet.

18.3 How NAT Works

When a device within the private network sends a request to a server on the Internet:

1. The packet is intercepted by the NAT.
2. The NAT replaces the private device's source IP address with its own public IP address.
3. The packet is forwarded to the destination server.

Upon receiving the response from the server:

1. The NAT intercepts the incoming packet.
2. Using a mapping table, the NAT replaces the destination IP address with that of the corresponding private device.
3. The packet is forwarded to the device inside the private network.

18.4 The Problem of Response Management

Traditional NAT allows sending requests to the outside, but initially does not handle responses correctly because internal devices use private IP addresses, which are not directly reachable from the Internet. When a server on the Internet sends a response, it is addressed to the NAT's public IP, which cannot know which internal device to return the packet to without a mapping mechanism. To resolve this limitation, an association table is introduced with the following fields.

When an internal device sends a request:

- The NAT records the private IP address, source port, and the corresponding public address with a unique temporary port in the table (**Port Address Translation - PAT**).
- When the response arrives at the public address and assigned port, the NAT consults the table and redirects the packet to the correct internal device.

18.5 Violation of the OSI Model: NAT and Layer Manipulation

NAT introduces an anomaly with respect to the layered structure of the OSI model:

- The OSI model mandates a clear separation between layers (e.g., the network layer should not interact with the transport layer).
- NAT, however, must modify information in both Layer 3 (IP) and Layer 4 (TCP/UDP), replacing not only IP addresses but also port numbers.
- This architectural "impurity" is necessary to enable NAT functioning, even though it breaks the abstraction principle between layers.

18.6 NAT Operations on Each Packet

Each packet passing through the NAT undergoes the following modifications:

1. **IP Layer (Layer 3):** The NAT replaces the source IP address with the network's public IP.
2. **Transport Layer (Layer 4 - TCP/UDP):** The NAT replaces the source port number with a unique number chosen by the router.
3. **Mapping Table:** The NAT registers the association between the private IP and port and the public IP and port.

Warning: The port number used by a private device might not be unique across different networks. The NAT dynamically assigns a unique NAT port. This process keeps connections active and guarantees correct routing of response packets.

18.7 NAT and Externally Accessible Servers

If one wishes to expose an internal server to the Internet, two solutions exist:

1. Assign a static public IP address to the server, making it directly accessible.
2. Utilize **Port Forwarding** (or **Destination NAT - DNAT**): the NAT forwards incoming traffic on a specific port of the public IP to an internal private IP.

Example: To make a web server accessible from the outside, the NAT is configured so that all traffic destined for the public IP on port 80 is forwarded to the private IP of the web server.

19 ARP (Address Resolution Protocol)

The **Address Resolution Protocol (ARP)** is a Network layer protocol (OSI Layer 3) used to determine the MAC address (Media Access Control, Layer 2) associated with a given IP address. When a device on a local area network (LAN) needs to send a packet to another device, it knows its IP address but needs the MAC address to actually transmit the Ethernet frame.

19.1 How Does It Resolve an IP to a MAC Address?

The source device sends an ARP request (**ARP Request**) in **broadcast** over the network, asking "Who has IP address X.X.X.X? Respond with your MAC address". The destination device, if present on the LAN, responds with an **ARP Reply** message in **unicast**, providing its MAC address. The sender updates its local **ARP table** with the received IP-MAC mapping, so it can directly send subsequent packets without repeating the ARP resolution.

19.2 Detailed Functioning of ARP

Each machine on a LAN has an ARP module handles requests and replies.

1. Checking the ARP Cache

- Before sending a packet, the device checks if it already has the MAC address of the destination IP in its ARP cache.
- If the entry exists and is valid, the device can send the frame directly.
- If it does not exist, or is expired, it proceeds with an ARP Request.

2. Sending the ARP Request

- The source device creates an ARP Request packet and sends it via broadcast (destination MAC: FF:FF:FF:FF:FF:FF).
- The packet contains the sender's IP and MAC and the recipient's IP.

3. Responding with ARP Reply

- The device with the requested IP receives the ARP Request and responds with an ARP Reply via unicast, including its MAC address.
- The sender updates its ARP cache and uses the received MAC address for subsequent packet transmissions.

4. ARP Cache and TTL

- Learned IP-MAC associations are stored in the ARP cache for a certain period of time (**TTL** – Time to Live).
- This avoids having to continuously repeat ARP requests for devices with which one communicates frequently.

19.3 ARP Packet Format

An ARP packet is encapsulated directly inside an Ethernet frame, with `EtherType 0x0806` (ARP).

19.4 ARP Across Different Networks: The Role of the Router (Gateway)

Attention: When the destination is outside the LAN, a traditional ARP Request would receive no response, because broadcast packets are not forwarded between different networks.

19.4.1 How is the Problem Resolved?

1. Net ID Calculation

- The machine performs a **bitwise AND** between its own IP and the Subnet Mask, obtaining its Net ID.
- It does the same with the destination IP.

2. Comparison of Net IDs

- If Net IDs match: the destination is on the same LAN, it can send the packet directly.
- If Net IDs differ: the destination is outside the LAN, so the packet must pass through the gateway (router).

3. The Gateway Steps In

- If Net IDs are different, the packet cannot be sent directly to the destination.
- Instead of making an ARP Request for the destination, the source machine makes an ARP Request for the **default gateway**.

4. Resolving the Gateway IP to a MAC

- If the default gateway's MAC is already known (ARP cache), the packet is immediately encapsulated in an Ethernet frame and sent to the router.
- If the default gateway's MAC is unknown, the device sends an ARP Request asking: "Who has the gateway IP? Respond with your MAC!"
- The router responds with an ARP Reply, providing its MAC.

5. Sending the Packet to the Router

- The source device encapsulates the IP packet in an Ethernet frame, using:
 - Source MAC = its own MAC
 - Destination MAC = gateway's MAC
- The router receives the packet and takes care of forwarding it toward the destination network (in proxy mode, the only difference is that the machine no longer needs to realize on its own that it must request the router's MAC, but the router itself understands that it should spoof the device's MAC).

19.4.2 Proxy ARP

There is a mechanism called **Proxy ARP**, used when a device needs to reach a destination outside the LAN but does not know it needs to go through a router.

- The router, seeing the ARP Request for an external IP, responds with its own MAC address, even though the requested IP belongs to another network.
- In this way, the sender believes that the destination IP is on the same LAN and sends packets to the router, which then forwards them correctly.
- Proxy ARP is less used today due to a clearer separation between networks and the widespread use of static and dynamic routing.

20 DHCP, ICMP, and Routing

20.1 IP Renewal and Release

IP addresses assigned by DHCP have a lease time (validity). The client attempts to renew the lease at specific times to avoid losing the address. If the lease expires, the client must repeat the entire **DORA** process to obtain a new one.

20.2 ICMP Format

ICMP (Internet Control Message Protocol) is a Layer 3 protocol used for diagnostics and error management. For example, a DHCP server uses an ICMP packet (*ping*) to verify that an IP address is not already in use before assigning it. ICMP has a *checksum* for the payload as well, unlike the IP protocol which features a checksum only for the header.

20.3 Routing vs. Forwarding vs. ARP

- **Routing:** The process of **dynamically populating** the routing table through specific protocols (e.g., RIP, OSPF), learning the network topology. It is a high-level logical process.
- **Forwarding:** The physical action of **forwarding** a packet from one interface to another based on information already present in the routing table.
- **ARP:** Finding MAC addresses given the IP (routing is instead used to find the best path).

WARNING: to send a packet one needs to know: **IP, MAC, and device port**. If one only had the MAC, the moment that device becomes a subnetwork, I would need to know all the MACs of all devices rather than just the IP address; furthermore, there is a different logic between MAC and IP.

Unlike Layer 2 forwarding, which relies on physical adjacency (MAC), Layer 3 routing handles routing between different networks.

21 Distance-Vector Routing Protocols

21.1 Principles of Operation

Distance-vector protocols (like **RIP**) are based on the idea that each node maintains a routing table, updating its network knowledge solely through information received from neighboring nodes. Periodically, each node:

1. Constructs its own adjacency table based on direct links.
2. Transmits its *distance vector* (a list of destinations and costs) to neighbors.
3. Uses neighbors' *distance vectors* to update its own table.

This mechanism allows a node to transition from local knowledge to a *global knowledge* of the network, albeit gradually. A node's routing table will contain:

- **Destination** (network or host address).
- **Next-hop** (the subsequent router).
- **Metric** (the cost or distance).

Note: "Global knowledge" does not mean the node has a complete map of the network; it means it knows how to reach every destination, relying on its neighbors for the correct path.

21.2 RIP (Routing Information Protocol)

RIP is a Distance-Vector-based protocol that uses the number of **hops** as its metric. Its maximum hop limit is 15, making it suitable only for small networks.

21.3 Issues with Distance-Vector Protocols

Asynchronous updating of *distance vectors* can lead to convergence problems.

21.3.1 Bouncing Effect

Occurs when nodes keep exchanging obsolete information due to delays. Costs bounce between different values, leading to erroneous propagation that can trigger the *Count to Infinity* problem.

21.3.2 Count to Infinity

Count to Infinity occurs when a destination becomes unreachable, but nodes continue to incrementally update the cost, mistakenly believing they can reach it via a neighbor, until reaching a maximum value (16 in RIP, indicating unreachability).

21.4 Solutions and Fixes

1. **Triggered Updates:** A mechanism that sends an immediate update as soon as a critical change (e.g., a failure) is detected, speeding up convergence.
2. **Split Horizon:** A technique that avoids advertising a route back to a neighbor if the route was learned from that neighbor. This rule prevents routing loops. Router C will not advertise its route to A back to B.
3. **Split Horizon with Poison Reverse:** A variant that, in case of a failure, advertises the route (learned from a neighbor) with an "infinite" cost, to prevent the neighbor from erroneously using it.

21.4.1 Limitations of Solutions: Why Triggered Updates and Split Horizon Can Fail

Even with these fixes, distance-vector protocols can fail in specific scenarios, such as the loss of update packets.

Failure Scenario:

1. **Link failure:** A link between two routers (e.g., B and C) breaks.
2. **Lost Triggered Update:** Router B detects the failure and sends a Triggered Update to its neighbors (A and D) to inform them that C is unreachable (infinite cost). Suppose the update message sent to D is lost.
3. **Propagation of incorrect information:**
 - Router D, not having received the update, continues to believe it can reach C through B and communicates this to its neighbors (e.g., E).
 - Router E receives the incorrect information from D and propagates it further.
4. **Triggering Count to Infinity:** The incorrect information re-enters circulation, reaching router B as well, which "learns" a new (wrong) route to C. The cost to reach C begins to increase with each information exchange, until reaching the maximum limit (16 for RIP), triggering the *Count to Infinity* problem.

This example demonstrates that losing a single update can negate countermeasures and lead to severe routing errors, making these protocols unreliable for complex networks.

22 Limits and Applications

22.1 Inapplicability on a Large Scale

Distance-vector protocols like RIP are **impossible** to use for large-scale routing (e.g., the Internet) due to three main reasons:

- **Scalability:** The routing table would need to contain an entry for every global network, an impractical number.
- **Slow Convergence:** Failures would take too long to propagate, causing widespread inefficiencies.
- **Bandwidth Consumption:** Periodically sending complete routing tables would consume an enormous amount of bandwidth.

Instead, the Internet uses more advanced and scalable protocols like **BGP** (Border Gateway Protocol), which is based on the concept of autonomous systems.

23 Link-State Routing Protocols

To overcome the limitations of distance-vector protocols, modern networks use **Link-State** protocols (e.g., OSPF), which offer faster convergence and higher reliability.

23.1 Principles of Operation (Link State)

In a Link-State protocol, each node constructs a complete map of the network topology through these steps:

1. **Neighbor discovery:** Each router uses ‘HELLO’ packets to identify directly connected routers (adjacencies).
2. **Propagation of link state:** Each router creates a packet called a **Link State Advertisement (LSA)** describing its direct links and their relative costs. This LSA is sent to all other routers in the network via a *controlled flooding* process.
3. **Map construction:** Each router collects all LSAs received from other routers and uses them to construct a complete and identical map of the network.
4. **Shortest path calculation:** Once the map is obtained, each router independently executes a shortest-path algorithm, such as **Dijkstra’s algorithm**, to calculate the best path (lowest cost) to each possible destination.

23.2 Advantages and Disadvantages of Link-State Protocols

- **Advantages:**
 - **Complete topology view:** Eliminates routing loops and the *Count to Infinity* problem.
 - **Fast convergence:** Information about topology changes is rapidly propagated via flooding, reducing reconfiguration times.
 - **Optimal path selection:** The use of Dijkstra’s algorithm guarantees the calculation of the best path based on the defined metric.
- **Disadvantages (Costs):**
 - **Network overhead:** Flooding LSAs can generate significant control traffic, especially in large networks.
 - **Computational load:** Running Dijkstra’s algorithm requires more CPU and memory resources than distance-vector protocols.

23.3 OSPF (Open Shortest Path First)

OSPF is the most widely adopted Link-State protocol. Its primary features include:

- **Hierarchical structure:** The network is divided into **Areas** to reduce update traffic and calculation complexity.
- **Advanced metrics:** The cost of a link can be based on metrics like bandwidth and latency, not just the hop count.
- **Load balancing (Multipath):** If multiple paths with the same cost exist toward a destination, OSPF can distribute traffic across them.

23.3.1 Structure of an OSPF Network: Areas

An OSPF network is organized hierarchically with a star structure.

- **Area 0 (Backbone Area):** It is the central area acting as the “backbone” of the network.
- **Other Areas:** All other areas must connect directly to Area 0. Communication between two non-backbone areas must always pass through Area 0.

This structure limits LSA flooding within a single area, improving scalability.

23.3.2 Optimizations in OSPF: Designated Router (DR)

To reduce control traffic in broadcast networks (like Ethernet), OSPF elects a **Designated Router (DR)** and a **Backup Designated Router (BDR)**.

- **Role of the DR:** Instead of exchanging LSAs with everyone, routers send their updates only to the DR. The DR then distributes the information to all other routers on the segment.
- **Advantage:** This mechanism drastically reduces the number of adjacencies and flooding traffic.

24 Large-Scale Routing and Modern Architectures

24.1 Routing Between Autonomous Systems: BGP

Routing on the Internet is based on the concept of an **Autonomous System (AS)**: a large network under a single administration (e.g., an ISP).

- **IGP Protocols (Interior Gateway Protocol)**: Like OSPF and RIP, they manage routing *inside* a single AS.
- **EGP Protocols (Exterior Gateway Protocol)**: Like **BGP (Border Gateway Protocol)**, they manage routing *between* different ASs (BGP is a distance-vector protocol but adds the list of hops the path has taken, eliminating the bouncing effect by filtering loops at individual stations [PATH VECTOR]).

Routers at the boundaries of an AS, called **Border Routers**, use BGP to exchange reachability information with other ASs, enabling global-scale communication.

WARNING: An Autonomous System (AS) is a network managed by a single entity, divided into OSPF areas for scalability reasons. Inside it, an IGP protocol (such as OSPF) handles routing: Area 0 (backbone) connects the other peripheral areas (such as Area 1, 2, etc.) which, in turn, contain the local networks. Routers connecting peripheral areas to Area 0 (called ABRs) use OSPF. A different protocol, EGP (such as BGP), is used by the boundary routers of the AS to connect the entire system to other ASs, forming the backbone of the Internet.

24.2 Evolution: Software Defined Networking (SDN)

The traditional distributed routing approach has been superseded in many modern networks by the **SDN (Software Defined Networking)** paradigm.

- **Separation Between Control Plane and Data Plane**: SDN separates routing logic (the "brain", or Control Plane) from the physical action of packet forwarding (the "arm", or Data Plane).
- **Centralized Controller**: Routing logic is moved from individual routers to a centralized **SDN controller** (often in the cloud).
- **Operation**:
 1. Routers send network status information to the controller.
 2. The controller calculates optimal routing tables for the entire network.
 3. The controller pushes the routing tables to the routers, which simply forward packets according to the received instructions.

SDN enables more efficient, flexible, and centralized network management, and is fundamental for technologies like 5G.

24.3 Transport Techniques: Tunneling

Tunneling is a technique that allows transporting packets of one protocol through a network that uses another.

1. **Encapsulation**: The original packet (e.g., IPv6) is "wrapped" inside another packet with a header compatible with the transit network (e.g., IPv4).
2. **Transport**: The encapsulated packet travels through the transit network.
3. **Decapsulation**: At the destination, the external header is removed and the original packet is restored.

This technique is essential for guaranteeing connectivity between networks with different protocols or for creating Virtual Private Networks (VPNs).

25 Border Gateway Protocol (BGP)

The Border Gateway Protocol (BGP) is the routing protocol used for communication between routers that manage traffic flow on the Internet. It is a Layer 3 (network layer) protocol dealing with addressing and routing, but its implementation occurs at the application layer and relies on TCP to guarantee reliable communication.

Main features

- BGP uses the TCP protocol on port 179 to establish reliable connections between routers.
- It maintains permanent TCP sessions between BGP units to exchange routing information in a stable and secure manner.
- It is primarily used to connect different Autonomous Systems (AS) within the Internet backbone, a global network distributed at an intercontinental level.

Route management and optimal path selection BGP handles the selection of the optimal path for packet transmission. Unlike other routing protocols, it does not use flooding algorithms but relies on a variant of Distance Vector Routing called Path Vector Routing.

25.1 Path Vector Routing and Bouncing Effect Management

Instead of simply transmitting route costs, as in traditional Distance Vector protocols (such as RIP), BGP includes additional path information known as Path Attributes. Each BGP node propagates a path vector containing the list of Autonomous Systems traversed to reach a given destination. This mechanism prevents the "bouncing effect" (routing loops), since an AS can reject paths containing itself in the list of traversed ASs, thus avoiding the reuse of a failed link.

Important If a link fails, BGP selects a new alternative path, even if it has a higher cost, thereby guaranteeing connectivity.

26 Problems with Traditional Routing

In routers, regardless of whether they have an internal routing function, operations exist that require processing the IP header, specifically the source address and destination address. These operations include table lookup to determine the packet's next hop. At critical points in the network where high performance is required, packet forwarding must be extremely efficient. However, classic table lookup based on IP addresses can be computationally expensive. To speed up the process, it is necessary to intervene at the algorithmic level, minimizing the processing required to determine a packet's path.

27 MPLS: Multi-Protocol Label Switching

To improve forwarding in the backbone area, MPLS (Multi-Protocol Label Switching) was introduced, a technology that speeds up the process by eliminating the need for IP-address-based lookups. Below are its features:

- *Switching instead of Routing*: MPLS adopts a switching logic, typically associated with Layer 2 of the OSI model, instead of performing IP-based forwarding (Layer 3).
- *Use of Labels*: Instead of analyzing the entire destination IP address, MPLS uses simple numerical labels to determine the packet's path.
- *Reduction of Computational Overhead*: Since routers do not need to perform a complex table lookup on an IP address, packet forwarding happens significantly faster.

28 How MPLS Works

The mechanism of MPLS can be divided into the following phases:

1. Traffic Labeling:

- Packets coming from peripheral areas and directed toward the backbone area are marked with an MPLS label.
- This numerical label is assigned according to the policies defined by the network operator.

- Boundary routers of the backbone area, known as ABRs (Area Border Routers), are responsible for adding the MPLS header (label) to incoming packets.

2. Label-Based Forwarding:

- Inside the backbone area, routers do not perform classic IP-based routing, but switch packets based exclusively on MPLS labels.
- This avoids complex IP lookups and speeds up the forwarding process.

3. Egress Decapsulation:

- When the packet reaches the edge router of the final destination, the egress ABR removes the MPLS header and restores the normal IP header.
- The original packet is then forwarded using normal IP routing to its final destination.

29 Label Management

Each router assigns and manages labels autonomously. It is possible to label not only individual packets but also entire traffic flows.

Notes For example, all traffic traveling from classroom Alpha to classroom Beta can be marked with the same label, allowing more efficient management. The key element homogenizing these flows is Quality of Service (QoS), tightly linked to the Type of Service (ToS) field present in IP packets.

30 Constructing Switching Tables

Switching tables are constructed to optimize the paths of packets with the same label, guaranteeing the best possible routing. In this context, the OSPF protocol is not eliminated but continues to play a fundamental role:

- It must collect link state information between routers in the network area.
- It must construct the network graph topology to identify shortest paths.

Instead of propagating traditional routing tables, the Designated Router handles the distribution of switching tables, facilitating label-based routing. This approach, known as Label Switching, speeds up network traffic compared to traditional IP Routing, improving efficiency and reducing the computational load on routers.

31 MPLS Packet Format

The composition of an MPLS header includes the following primary fields:

- *Label*: Functions as an identifier and allows label-based packet routing instead of traditional IP address-based routing.
- *Class of Service (CoS)*: Allows classifying the packet based on Quality of Service (QoS). This enables the packet and all those belonging to the same flow to move along a specific path, guaranteeing priority and optimal traffic management.
- *S (Bottom of Stack)*: Indicates whether the MPLS packet is part of a stack of labels (Multiple Label). If the value is 1, the packet represents the last label in the stack.
- *TTL (Time To Live)*: Serves to prevent infinite loops in the network. Each router decrements the packet's TTL value, and if it reaches 0, the packet is discarded.

32 Spanning Tree

The Spanning Tree is a technique used in constructing the shortest-path tree within a complex network. It is employed particularly in networks with Link-State protocols, where each node can build the network graph and calculate shortest paths to other nodes.

Port types in the spanning tree

1. *Root Port (RP)*: Port pointing toward the root of the tree.

2. *Designated Port (DP)*: Port leading toward leaf nodes.

3. *Blocked Port (BP)*: Port disabled to prevent the formation of loops within the network.

Operation Upon receiving a packet on the Root Port, the router propagates it to all Designated Ports. This process builds a shortest-path tree, optimizing the number of packets sent in a broadcast transmission, thereby reducing network congestion. The Spanning Tree algorithm is designed to solve two main problems in computer networks:

- *Avoid network overload*: Prevents redundant transmission of packets to neighboring nodes, optimizing routing.
- *Prevent infinite loops*: Impedis a packet from getting trapped in a loop between two or more nodes, ensuring efficient traffic management.

33 Quality Of Service (QoS) in Communication Networks

Traditional networks operate with a **best-effort** approach, offering no guarantees on transmission quality. To overcome this limitation, the concept of **Quality of Service (QoS)** is introduced, allowing network traffic management and differentiation based on specific application needs.

The main requirements to meet are:

- **Sensitivity to delay (Latency)**: Minimum time for transmission, crucial for real-time applications (e.g., VoIP, online gaming).
- **Required bandwidth (Bandwidth)**: Amount of data transmittable per unit of time, fundamental for high-definition video streaming.
- **Sensitivity to Jitter**: Variation in packet arrival delay. High jitter degrades audio and video quality.

To mitigate jitter, streaming applications use a **playout buffer**, which stores incoming data before playing it back, ensuring a steady stream.

33.1 Queue Management: Input Queuing

Congestion occurs when incoming traffic to a router exceeds its processing capacity, leading to packet loss (**buffer overflow**). To manage this problem, active queue management techniques are used.

33.1.1 Random Early Detection (RED)

RED is a proactive congestion management algorithm that discards packets *before* the queue saturates completely.

- **Principle of operation**: Calculates an average queue occupancy level (u).
- **Operating thresholds**:
 - **Minimum threshold**: Below this threshold, no packets are dropped.
 - **Maximum threshold**: Above this threshold, all incoming packets are dropped.
 - **Between the two thresholds**: Packets are dropped with a probability that increases linearly as queue occupancy increases.
- **Advantages ("Early" component)**:
 - Prevents complete buffer saturation.
 - Signals congestion in advance to transport protocols (e.g., TCP), which can reduce their transmission speed.
 - Avoids total network lockup.

33.1.2 ICMP Choke Packets

Another technique (less used) involves sending ICMP packets (**choke packets**) backward toward the source to signal congestion. However, it presents drawbacks, such as increasing control traffic and delaying reaction time.

33.2 Queue Management: Output Queuing

In output queue management, a **classifier** sorts incoming packets into different queues based on their *Type of Service (ToS)* field. A **scheduler** then decides which queue to serve for transmission.

33.2.1 The Starvation Problem

A simple strict-priority scheduler (which always serves the highest priority queue) can lead to **starvation**: low-priority queues might never be served if there is a constant flow of high-priority traffic.

33.2.2 Weighted Fair Queuing (WFQ) as a Solution

To avoid starvation, **Weighted Fair Queuing (WFQ)** is used. Each queue is assigned a **weight** (w_i), and the scheduler allocates bandwidth proportionally to those weights.

- The bandwidth fraction for queue i is calculated as:

$$\frac{w_i}{\sum_j w_j}$$

- In this way, no queue is completely excluded (*starvation-free*).

Example: With 3 queues of weights $w_1 = 4, w_2 = 2, w_3 = 2$, the bandwidth is allocated at 50

33.2.3 Weighted RED (WRED)

Combining WFQ and RED yields **WRED**. This technique applies different RED thresholds to queues with different priorities. Lower priority packets are dropped sooner, protecting more critical traffic during congestion.

33.3 Traffic Shaping

Traffic Shaping is a technique that regulates traffic flow *before* it enters the network, ensuring it complies with agreed parameters.

- **Controlled parameters:**
 - Average packet rate.
 - Maximum traffic peak.
 - Maximum burst size (packet burst).

33.3.1 Token Bucket

It is the most common mechanism to implement traffic shaping.

1. A "bucket" (*bucket*) is filled with "tokens" (*token*) at a constant rate.
2. To transmit a packet, a token must be "spent".
3. If the bucket is empty, the packet must wait for a new token to be generated.

This mechanism enforces a limit on the average transmission rate and allows managing traffic bursts (up to the maximum capacity of the bucket, so it does not manage individual flow traffic but total traffic).

33.3.2 Call Admission Control (CAC)

This policy decides whether to accept or reject a new traffic flow based on available network resources, ensuring that the QoS of existing flows is not compromised. It guarantees that only traffic conforming to contractual parameters is accepted.

33.4 Router Protocol Structure

1. Initial CAC to filter connections and deny them if they cannot be supported
2. Traffic shaping (token bucket)
3. WRED (additional control or useful if a flow for some reason is not subject to traffic shaping)

34 IPv6: Features and Transition from IPv4

IPv6 is the evolution of IPv4, designed to solve the problem of address exhaustion and to improve the efficiency and security of the protocol.

34.1 Main Features and Differences with IPv4

- **128-bit Addresses:** Solves the address scarcity problem (from 2^{32} to 2^{128}).
- **Simplified Header:** The IPv6 header is leaner and has a fixed number of fields. Optional fields are managed via *Extension Headers*.
- **Traffic Class and Flow Label:** Fields to support QoS and to identify packet flows requiring the same type of handling.
- **Next Header:** Field indicating which header follows the IPv6 one (e.g., TCP, UDP, or an extension header), creating a flexible chain of headers.
- **No fragmentation by routers:** Packet fragmentation, if necessary, is handled only by the source host, reducing the workload on intermediate routers.

34.1.1 Address Format and Hierarchical Structure

IPv6 addresses are designed for efficient hierarchical routing, reducing the size of global routing tables. The structure is often split into:

- **Global routing prefix:** Assigned to an ISP or a large organization.
- **Subnet ID:** Used to define subnetworks within the organization.
- **Interface ID:** Identifies a specific device within the subnetwork.

34.2 Compatibility and Transition Mechanisms

The transition from IPv4 to IPv6 is a gradual process requiring co-existence mechanisms.

34.2.1 Dual Stack

A device (host or router) implements both protocol stacks, IPv4 and IPv6. It can therefore communicate natively with IPv4 hosts using IPv4 and with IPv6 hosts using IPv6. It is the most complete solution but requires supporting both protocols across the entire infrastructure.

34.2.2 Tunneling

Allows two IPv6 hosts to communicate through an IPv4 network.

1. A boundary router receives an IPv6 packet.
2. **Encapsulates** the IPv6 packet inside an IPv4 packet.
3. The IPv4 packet traverses the intermediate network.
4. The destination boundary router **decapsulates** the packet, extracting the original IPv6 packet.

34.2.3 Translation (NAT64)

A gateway translates packet headers from IPv6 to IPv4 and vice versa, allowing an IPv6-only host to communicate with an IPv4-only host. This solution is more complex and can introduce compatibility issues with certain protocols.

35 TCP Protocol (Transmission Control Protocol)

The **Transmission Control Protocol (TCP)** is a connection-oriented transport layer protocol providing a **reliable** and **ordered** data transfer service between application processes. Its architecture is designed to handle separate communication flows, correct errors, and control data flow.

35.1 Managing Multiple Connections on a Server

A server (e.g., a web server on port 80) must be able to handle multiple clients simultaneously. To do so, it adopts a mechanism based on child processes and dynamic ports:

1. **Connection acceptance and fork()**: When a client connects to the well-known port (e.g., 80), the server accepts the connection and creates a dedicated **child process** via the 'fork()' system call.
2. **Dynamic port allocation**: The child process does not continue using port 80. The operating system assigns it a new **ephemeral port** (above 1024) for communication with that specific client.
3. **Traffic redirection**: Port 80 on the server immediately becomes free again to accept new connections. The operating system maintains a table mapping the client's connection (identified by source IP and port) to the child process's new dynamic port.

In this way, the server can handle hundreds of simultaneous connections without conflicts.

35.2 Connection Process: Socket Primitives

TCP communication relies on the use of **sockets**, which represent communication endpoints.

35.2.1 Client Side

1. **DNS Resolution**: The client obtains the server's IP address via a DNS request.
2. **Socket Creation**: It uses the 'socket()' primitive to obtain a file descriptor for communication.
3. **Connection**: Invokes 'connect()', specifying the server's IP and port. This call initiates the *three-way handshake*, and the client process blocks waiting for the connection to be established.

35.2.2 Server Side

1. **Socket Creation**: The server creates a socket with 'socket()'.
2. **Binding**: Uses 'bind()' to associate the socket with an IP address and a specific port (e.g., 80).
3. **Listening**: With 'listen()', the server enters listening mode and defines the queue size for pending connections.
4. **Acceptance**: The 'accept()' call blocks the server until a connection request arrives from a client. When it arrives, 'accept()' returns a new socket dedicated to that specific connection.

35.2.3 Data Exchange

Once the connection is established, both endpoints use the 'send()' and 'receive()' primitives to exchange data bidirectionally.

35.3 TCP Header

The TCP header has a minimum size of 20 bytes and contains crucial fields to guarantee reliable communication.

- **Source Port and Destination Port** (16 bits each): Identify the sender and recipient application processes.
- **Sequence Number** (32 bits): Since TCP is a *byte-stream* protocol, this field indicates the sequence number of the **first byte** of data in the segment.
- **Acknowledgment Number** (32 bits): Indicates the sequence number of the **next byte** that the recipient expects to receive. Functions as receipt confirmation.
- **Control Flags** (e.g., SYN, ACK, FIN, RST): Control bits to manage connection phases (opening, closing, reset).
- **Window Size** (16 bits): Crucial field for **flow control**. Indicates the amount of space (in bytes) available in the sender's receive buffer for this segment.
- **Checksum** (16 bits): Used to verify header and data integrity.

35.3.1 Pseudo-Header and Checksum

To guarantee that a TCP segment has arrived at the correct destination (and has not, for example, been incorrectly rerouted), the checksum is calculated on a virtual structure called a **pseudo-header**, which includes:

- Source and destination IP addresses.
- Protocol code (6 for TCP).
- Length of the TCP segment.

This mechanism binds the portfolio TCP segment to the IP packet transporting it, increasing reliability.

35.4 Opening Phase: Three-Way Handshake

The TCP connection is established through a three-step procedure:

1. **Client → Server (SYN)**: The client sends a segment with the **SYN** (Synchronize) flag active and a random initial sequence number (e.g., 'SEQ=x').
2. **Server → Client (SYN-ACK)**: The server responds with a segment having both the **SYN** and **ACK** flags active.
 - Its sequence number is a new random value (e.g., 'SEQ=y').
 - The acknowledgment number is 'ACK=x+1', confirming receipt of the client's first segment.
3. **Client → Server (ACK)**: The client sends a final segment with the **ACK** flag active to confirm receipt of the server's SYN. The acknowledgment number is 'ACK=y+1'.

At this point, the connection is established (*ESTABLISHED*) and data transfer can begin.

35.5 Flow Control and Channel Utilization

Flow control prevents a fast sender from overwhelming a slow receiver.

- **Sliding Window**: The sender can transmit an amount of data equal to the **Window Size** communicated by the receiver.
- **Dynamic Update**: As the receiver processes data from its buffer, it sends ACKs with an updated 'Window Size' value, communicating the free space.
- **Zero-Window**: If the receiver communicates a 'Window Size = 0', the sender must stop.

35.5.1 Stall Management: Persist Timer

If an ACK with a 'Window Size' update (from 0 to a positive value) is lost, the sender could remain blocked indefinitely. To avoid this:

- When the sender receives 'Window Size = 0', it starts a **Persist Timer**.
- Upon timer expiration, it sends a small **probe** packet to request a window status update. This guarantees communication does not stall.

35.6 Idle Connection Management: Keep-Alive Timer

To detect whether a connection is still active even in the absence of traffic, the **Keep-Alive Timer** is used.

- If a connection remains idle for a long period, the server begins periodically sending **probe** packets.
- If it receives no response after a certain number of attempts, the server assumes the client disconnected abnormally and closes the connection, freeing resources.
- Configuring this timer is a compromise: a value too low can close valid but slow connections, while one too high wastes server resources.

36 Congestion Control in TCP

36.1 Retransmission Mechanisms: RTO vs. Fast Retransmit

Both mechanisms detect packet loss but react differently.

- **Retransmission Timeout (RTO):**
 - **Trigger:** Expiration of a waiting timer for an ACK.
 - **Meaning:** Signal of a potentially serious problem (lost segment, lost ACK, or severe delay).
 - **Action:** The sender retransmits the unconfirmed segment. It is a fundamental but slow safety mechanism.
- **Fast Retransmit:**
 - **Trigger:** Receipt of **three duplicate ACKs** for the same sequence number.
 - **Meaning:** Strong signal that a single segment was lost, but the network is still functioning (subsequent packets are arriving).
 - **Action:** The sender retransmits the lost segment *immediately*, without waiting for the RTO to expire.

36.2 Flow Optimization: Nagle's and Clark's Algorithms

These **self-adaptive** algorithms reduce overhead in asymmetrical communication scenarios. Indeed, when a packet is sent by the sender (for example, a character to write on a remote terminal), the receiver sends 3 other messages:

- ACK (Acknowledgment) to confirm character receipt
- Window Size (WS) to update the receive window
- Echo of the character to display it on the remote terminal

36.2.1 To avoid clogging the bandwidth for every individual character, these 2 methods are used:

- **Nagle's Algorithm (Sender side):**
 - **Problem:** Slow sender transmitting many small packets (high overhead).
 - **HOW:**
 - * If there are no previously sent packets in transit, it dispatches the packet immediately.
 - * If there are already packets in transit, it groups all data to be sent into a single packet, which will be sent as soon as the ACK for the previous one is received.
- **Clark's Algorithm (Receiver side):**
 - **Problem:** Slow receiver freeing the buffer a few bytes at a time, causing short window updates and inducing the sender to transmit tiny packets.
 - **HOW:**
 - * Instead of sending an ACK (containing the WS) every time space is freed (starting from a full buffer scenario), it waits for a time interval or until a significant portion of the buffer is empty.
- **SELF-CLOCKING:** This is the method automatically implemented in TCP that uses both Nagle's and Clark's concepts.

36.3 Congestion Control (Congestion Window - cwnd)

TCP adapts its transmission speed not only to the receiver but also to the network status, using a congestion window ('cwnd'). Its value is adjusted dynamically.

36.3.1 Window Growth Phases

- **Slow Start:** At connection startup. The 'cwnd' starts at 1 MSS (one packet at a time) and grows **exponentially** (doubles every RTT). The goal is to quickly find network capacity. This phase lasts until 'cwnd' reaches the threshold 'ssthresh'.
- **Congestion Avoidance:** Once the 'ssthresh' threshold is surpassed. 'cwnd' growth becomes **linear** (increases by 1 MSS per RTT), probing the network with more caution to avoid creating congestion.

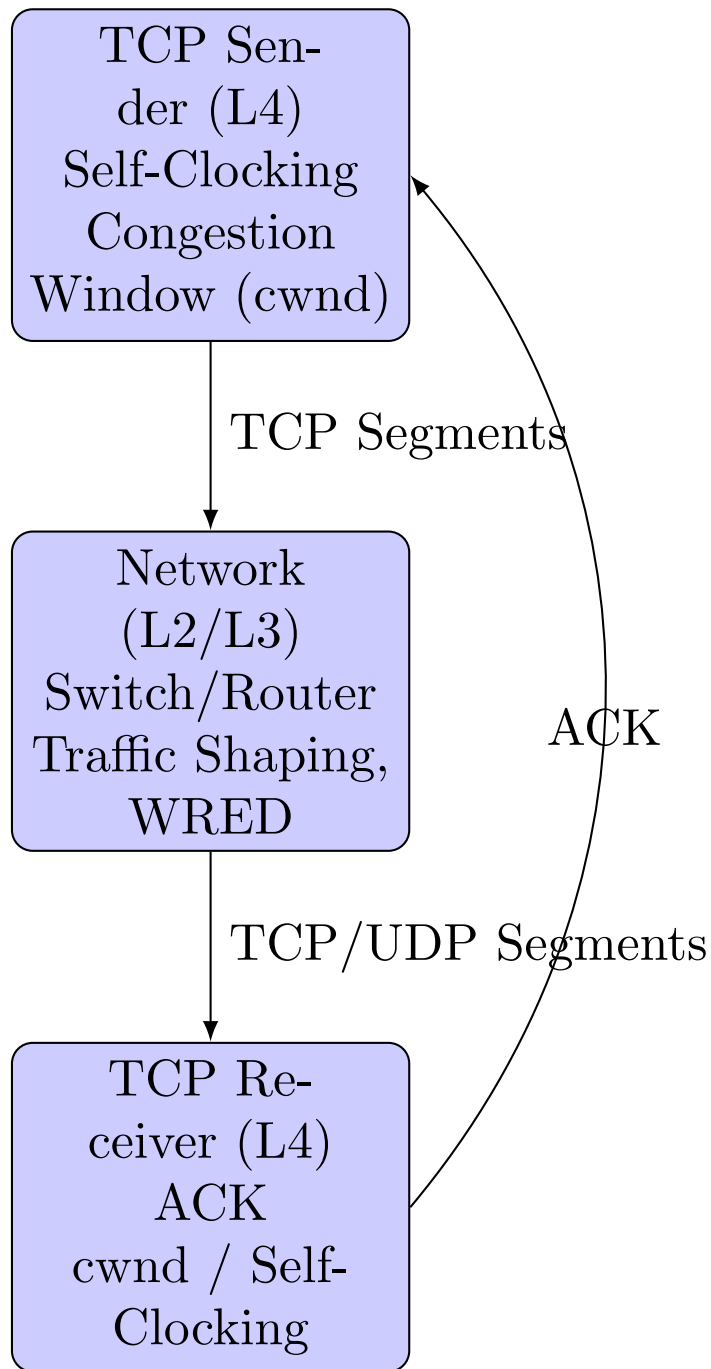
36.3.2 Reaction to Packet Loss (Congestion)

Loss is the main signal of congestion. TCP's reaction varies depending on the signal's severity.

- **Reaction to Timeout (RTO):** Signal of **severe congestion**.
 - 'ssthresh' is halved.
 - 'cwnd' is reset to **1 MSS**.
 - Transmission restarts from the **Slow Start** phase.
- **Reaction to Fast Retransmit (3 duplicate ACKs):** Signal of **mild congestion**.
 - 'ssthresh' is halved.
 - 'cwnd' is set to the new 'ssthresh' value (instead of 1).
 - Transmission resumes directly from the **Congestion Avoidance** phase (**Fast Recovery** mechanism).

36.3.3 Self-Clocking vs. cwnd

- SELF-CLOCKING deals with preventing clogging between the sender and receiver.
- CWND deals with preventing network clogging.



37 Advanced Mechanisms in TCP

To optimize performance and reliability, TCP utilizes sophisticated algorithms for managing timers, retransmissions, and connection teardowns.

37.1 Dynamic Calculation of Timeout (RTO)

The efficiency of TCP relies on a well-calibrated **Retransmission Timeout (RTO)** value. TCP computes it dynamically to adapt to network conditions.

- **Goal:** Avoid premature retransmissions (RTO too low) or unnecessary waiting (RTO too high).
- **Method:** It relies on an estimation of the Round Trip Time (RTT) and its variance (deviation), calculated via weighted averages to smooth out instantaneous fluctuations.
 - **Smoothed RTT:** $RTT_{new} = \alpha \cdot RTT_{old} + (1 - \alpha) \cdot M$ (new RTT measurement)
 - **Smoothed Deviation:** $D_{new} = \alpha \cdot D_{old} + (1 - \alpha) \cdot |M - RTT_{new}|$
 - **Final RTO Calculation:** $RTO = RTT_{new} + 4 \cdot D_{new}$ (4 to leave room for potential delays)
- **Initial Cases:**
 - **First segment:** RTO is a default value (e.g., 3 seconds).
 - **Second segment:** $RTO = M + 4 \times (\frac{M}{2})$ (since D and RTT are zero initially).
 - **From the third onwards:** The complete formulas are used.

37.2 Karn's Rule: Handling Ambiguity in Retransmissions

- **Problem:** When a segment is retransmitted, the incoming ACK is ambiguous: does it refer to the first or second transmission? Using this measured RTT can lead to completely erroneous RTO estimations.
- **Solution (Karn's Rule):**
 1. **Ignore the measurement:** If a segment has been retransmitted, the RTT measurement for that ACK is ignored.
 2. **Apply Exponential Backoff:** Instead of recalculating the RTO, it is doubled: $RTO_{new} = 2 \cdot RTO_{old}$. This conservative approach helps handle congestion (it only serves to ignore the measurement if the packet is retransmitted, not to measure it correctly, and to guarantee the necessary time for the packet).

37.3 Improving Reliability: Timestamps and SACK

- **TCP Timestamp Option:** To overcome Karn's ambiguity, the Timestamp option allows the sender to include a timestamp in the segment. The receiver echoes this timestamp in the corresponding ACK; the sender can then map the ACK to its corresponding segment and compute the total round-trip time accurately.
- **Selective Acknowledgment (SACK):**
 - **Problem with Cumulative ACKs:** If packet 5 is lost but packets 6, 7, and 8 arrive, the cumulative ACK will keep requesting 5, failing to inform the sender that subsequent packets have already arrived successfully.
 - **SACK Solution:** The receiver, along with the cumulative ACK, can specify non-contiguous data blocks received correctly (e.g., "I received blocks 6-8"). This allows the sender to retransmit **only the missing segments**, drastically improving efficiency in case of multiple losses.

37.4 Connection Teardown (Four-Way Handshake)

Closing a TCP connection is a four-step process to ensure both sides have completed transmission.

1. **Host A → Host B (FIN):** The host initiating the closure (active close) sends a segment with the FIN flag.
2. **Host B → Host A (ACK):** Host B confirms receipt of the FIN. From this moment, host A can no longer send data, but B can (*half-close*).
3. **Host B → Host A (FIN):** When B has finished transmitting its data as well, it sends its own FIN segment.
4. **Host A → Host B (ACK):** Host A confirms B's FIN and enters the **TIME-WAIT** state.

TIME-WAIT State: Crucial state where the host that initiated the closure remains. It lasts for a defined period (e.g., 2 minutes) to ensure any duplicate packets from the old connection are discarded and do not interfere with a new connection.

38 UDP (User Datagram Protocol)

UDP is an alternative transport protocol to TCP, characterized by its simplicity and speed at the expense of reliability.

38.1 Main Features

- **Connectionless:** There is no connection opening or closing handshake. Datagrams are sent directly.
- **Best-Effort:** Offers no guarantee on delivery, ordering, or absence of duplicates. It implements neither flow control nor congestion control.
- **Minimum Overhead:** The header is only 8 bytes, making it extremely lightweight and efficient.
- **Optional Checksum:** Uses a pseudo-header (like TCP) to include IP addresses in the checksum calculation, but its use is optional.

38.2 When to Use UDP?

UDP is the ideal choice for applications where **speed** and **low latency** are more important than absolute reliability. Managing potential losses is delegated to the application.

- **Audio/Video Streaming** (e.g., VoIP, video conferencing): Losing an individual packet is preferable to introducing delays for retransmission.
- **Online Gaming:** Minimum latency is critical.
- **Fast Query Services** (e.g., DNS): Requests and responses are small and must be quick.

39 Real-Time Protocols: RTP and RTCP

For the transmission of multimedia data (audio/video), which require low latency more than absolute reliability, a pair of specific protocols is used: RTP for data and RTCP for control.

39.1 General Features

- **Data/Control Separation:** RTP (Real-time Transport Protocol) transports multimedia streams, while RTCP (RTP Control Protocol) handles monitoring and session statistics.
- **Based on UDP:** Both protocols operate over UDP to minimize latency, sacrificing the intrinsic reliability of TCP (no automatic retransmissions).
- **Operation in Pairs:** Each RTP stream is always associated with an RTCP stream on the subsequent port.

39.2 RTP (Real-time Transport Protocol) - The Data Flow

Its task is to encapsulate multimedia data and provide the information necessary for its correct playback.

- **RTP Header - Key Fields:**
 - **Sequence Number:** Numbers the packets to allow the receiver to reorder them and detect any losses.
 - **Timestamp:** Indicates the sampling instant of the first byte of the packet. It is fundamental for synchronization and for calculating *jitter* (delay variation).
 - **Payload Type (PT):** Identifies the type of encoding used (e.g., G.711 for audio, H.264 for video).
 - **Synchronization Source (SSRC):** A unique numerical identifier for each stream source within a session.

39.3 RTCP (RTP Control Protocol) - The Control Flow

RTCP operates in parallel with RTP to provide feedback on transmission quality.

- **Main Functions:**
 - **QoS Monitoring:** Receivers periodically send reports to the sender with statistics on lost packets, jitter, and Round-Trip Time.
 - **Synchronization:** Provides data (via NTP timestamps) to synchronize different streams (e.g., audio and video from the same source).
 - **Feedback and Adaptation:** Based on RTCP reports, an intelligent sender can adapt the stream quality (e.g., changing the codec or bitrate) according to network conditions.

40 DNS (Domain Name System)

DNS is the distributed system that translates human-readable domain names (e.g., `www.google.com`) into numerical IP addresses (e.g., `142.250.184.68`), required for routing packets across the network. It operates at the application layer (Layer 7).

40.1 Hierarchical Structure and Name Resolution

- **Tree Structure:** DNS has a hierarchical structure that starts from a root (*root*) and branches into Top-Level Domains (TLD, e.g., `.com`, `.it`), second-level domains (e.g., `google`, `unimi`), and subdomains.
- **Authoritative DNS Servers:** For each domain, at least one authoritative server exists that holds the official records for that domain (e.g., `.edu`, `.com`, `.org`, etc.).
- **Recursive vs. Iterative Resolution:**
 - **Recursive:** The client asks its local DNS resolver to find the IP. The resolver does all the work, contacting other necessary DNS servers, and returns the final answer to the client.
EG: my PC asks its local DNS server, which (if it doesn't have the answer) asks the root DNS, which asks the TLD, and so on; once the response is received, the process is reversed and replies back to the preceding DNS down to my PC.
 - **Iterative:** The local resolver asks the root server, which redirects it to the TLD server (A TLD [Top-Level Domain] is the final part of a domain name, the one following the last dot. Think of `.com`, `.it`, `.org`). The resolver asks the TLD server, which redirects it to the authoritative server, and so forth, until the response is obtained.
EG: my PC asks its local DNS server, which provides it with the root DNS IP, then my PC contacts the root DNS, which gives it the IP of a TLD, and so on.
- **Caching:** To speed up requests, DNS servers temporarily store (in a **cache**) the answers obtained. The duration of stay in cache is defined by the record's **TTL** (time to live) field.

40.2 DNS Records

The DNS database is composed of a cache table that stores past requests for a specified time (to avoid making the same queries repeatedly) populated by **Resource Records (RR)**, which map a name to a value. The most common types are:

- **A:** Maps a name to an **IPv4** address.
- **AAAA** ("quad-A"): Maps a name to an **IPv6** address.
- **MX** (Mail Exchange): Specifies the mail server responsible for a domain.
- **CNAME** (Canonical Name): Creates an alias, mapping one name to another name.
- **NS** (Name Server): Indicates which DNS server is authoritative for a domain.
- **TXT:** Allows entering arbitrary text, frequently used for security mechanisms (e.g., SPF).

41 Email Architecture

The email service is a distributed system for exchanging messages, based on standardized components and protocols.

41.1 Key Components

- **User Agent (UA):** The email client (e.g., Outlook, Gmail) used to compose and read emails.
- **Message Transfer Agent (MTA):** The mail server responsible for routing and transferring messages between different domains.
- **Message Store:** The server space where emails are archived waiting to be read by the recipient.

41.2 Email Sending and Receiving Flow

1. **Sending from the Client:** The user writes an email using their UA and sends it to their MTA server (often via SMTP).
2. **Server-to-Server Transfer (SMTP):** The sender's MTA contacts the recipient's MTA on port 25 and transfers the message using the **SMTP (Simple Mail Transfer Protocol)** protocol.
3. **Archiving:** The recipient's MTA receives the email and deposits it in the user's Message Store.
4. **Reading by the Recipient:** The user connects to their mail server using their UA to read the email.

41.3 Protocols for Mail Access

For the reading phase, the user uses one of these two protocols:

- **POP3 (Post Office Protocol v3):** Downloads emails from the server to the local device, typically **removing them** from the server. Suitable for access from a single device.
- **IMAP (Internet Message Access Protocol):** Manages emails **directly on the server**. Modifications (reading, deletion, moving into folders) are synchronized across all devices. This is the standard for multi-device access.

41.4 Attachment Management: MIME and Base64

- **The Problem:** SMTP is a textual protocol using 7-bit ASCII encoding for characters, with special bytes for control commands (e.g., line breaks, spaces, etc.), called **NVT (Network Virtual Terminal)**. SMTP/NVT cannot directly transport binary data like images, videos, or PDFs.
- **The Solution - MIME (Multipurpose Internet Mail Extensions):** This is an extension of SMTP that adds headers to describe non-textual content without modifying the underlying protocol.
 - **Content-Type:** Specifies the file type (e.g., `image/jpeg`).
 - **Content-Transfer-Encoding:** Indicates how binary data was encoded to be transmitted as text.
- **The Encoding - Base64:** This is the most common algorithm used by MIME. It converts binary data into a sequence of printable ASCII characters, making them safe for transmission via SMTP.
 - Binary data is processed in blocks of 3 bytes (24 bits) and divided into 4 blocks of 6 bits each.
 - Each 6-bit block is converted into an ASCII character using the Base64 table (64 possible characters).
 - If the data is not a multiple of 3 bytes, padding with the `=` character is used to complete the group.

41.5 SMTP Structure

- Everything is written in NVT format.
- Header + message (textual only).
- Images are added to the header as an extension.

42 FTP (File Transfer Protocol)

FTP is a standard protocol for transferring files between computers on a network, based on TCP to guarantee reliability.

42.1 Dual-Connection Architecture

The distinctive feature of FTP is the use of two separate communication channels:

- **Control Connection (server port 21):**
 - Persistent channel, remains open throughout the session.
 - Used to send commands (e.g., ‘USER’, ‘PASS’, ‘LIST’, ‘GET’) and receive server responses.
- **Data Connection (server port 20 - WARNING: only in active FTP connections):**
 - Temporary channel, opened only for the actual transfer of a file or a directory listing.
 - Closed at the end of each transfer.

42.2 Data Connection Modes: Active vs. Passive

(They perform the same task, but passive is more widely used because it resolves firewall issues).

- **Active FTP:**
 1. The client connects to port 21 of the server (control).
 2. When transferring data, the client communicates a port of its own to the server to listen on.
 3. The **server initiates the data connection** from its port 20 toward the port specified by the client.
 4. *Problem:* Frequently blocked by client-side firewalls.
- **Passive FTP:**
 1. The client connects to port 21 of the server (control).
 2. When transferring data, the client sends the ‘PASV’ command.
 3. The server responds by indicating a port of its own (not port 20) on which it listens.
 4. The **client initiates the data connection** toward the port specified by the server.
 5. *Advantage:* Highly compatible with firewalls.

42.3 Separation Between Control Plane and Data Plane

FTP’s architecture is a classic example of plane separation:

- **Control Plane:** The connection on port 21, handles session logic (authentication, commands).
- **Data Plane:** The temporary connection, strictly handles the transport of the file’s bits.

43 IP Allocation Phases in DHCP

The **DHCP** (Dynamic Host Configuration Protocol) protocol allows clients to automatically obtain an IP address and other network parameters. Allocation takes place in four primary phases:

1. DHCP Discover:

- The client, not knowing a DHCP server, sends a broadcast message on the local network to discover available servers.
- This message contains the client's MAC address and other optional information.

2. DHCP Offer:

- DHCP servers receive the Discover request and respond with an *Offer* message, proposing an available IP address and configuration parameters (subnet mask, gateway, DNS, lease time).
- The Offer is sent as broadcast or unicast to the client.

3. DHCP Request:

- The client selects an offer among those received and sends a *Request* message to all DHCP servers, indicating which offer it accepts.
- This also serves to inform other servers not to allocate that IP address to other clients.

4. DHCP Acknowledge (ACK):

- The DHCP server that received the Request confirms the allocation with an *ACK* message.
- The client can now configure its network interface with the IP address and other parameters provided.
- The lease has a defined duration, at the end of which the client must renew it.

44 HTTP (HyperText Transfer Protocol)

HTTP is the application-layer protocol underpinning the World Wide Web. It operates on a **request-response** (request-reply) model: a client requests a resource (e.g., a web page) and a server responds by providing it. Its evolution has focused on resolving bottlenecks to improve speed and efficiency.

44.1 HTTP/1.0: Non-Persistent Connections

The first version of HTTP used a simple but inefficient model.

- **Operation:** For each single object requested (e.g., an image, a CSS file), a new TCP connection was opened and closed immediately after the response.
- **Primary Problems:**
 - **High Overhead:** Continuously opening and closing TCP connections (with their associated three-way handshake) consumed significant server resources and introduced latency.
 - **TCP Inefficiency:** Connections, being very short, never fully benefited from TCP's congestion control mechanisms (always operating in the *slow start* phase).

44.2 HTTP/1.1: Persistent Connections and Pipelining

HTTP/1.1 introduced two fundamental improvements to solve the problems of version 1.0.

- **Persistent Connections (Keep-Alive):** The TCP connection is kept open for multiple consecutive requests, amortizing handshake costs and improving performance.
- **Pipelining:** The client can send multiple requests sequentially on the same connection without waiting for the response to the previous one.
- **New Problem - Head-of-Line (HoL) Blocking (Layer 7):** If the first request in the pipeline is slow to process, all subsequent responses are blocked, even if they are already ready.

44.3 HTTP/2: Multiplexing over TCP

HTTP/2 revolutionized request handling to resolve the HoL blocking of HTTP/1.1.

- **Multiplexing:** Multiple request-response streams (*stream*) are transmitted in parallel over the **same single TCP connection**. A slow response in one stream does not block the others.
- **Other Improvements:**
 - **Server Push:** The server can proactively send resources it knows the client will require shortly (e.g., a page's CSS and JS files).
 - **Header Compression (HPACK):** Reduces redundancy and the size of HTTP headers.
- **New Problem - Head-of-Line Blocking (Layer 4, TCP):** Although the streams are independent at the application layer, they are all encapsulated in a single ordered TCP stream. Losing a single TCP packet blocks the delivery of **all streams** until that packet is retransmitted.

44.4 HTTP/3: Shifting to QUIC over UDP

To overcome the intrinsic limitation of TCP, HTTP/3 radically changes the underlying transport protocol.

- **Based on QUIC (Quick UDP Internet Connections):** HTTP/3 does not use TCP, but QUIC, a new transport protocol operating over UDP.
 - **Advantages of QUIC:**
 - **Resolves TCP's HoL Blocking:** QUIC manages streams natively and independently. Losing a packet related to one stream does not block the others.
 - **Faster Handshake (0-RTT):** Drastically reduces the time needed to establish or resume a secure connection.
 - **Integrated Security:** Encryption (TLS 1.3) is natively integrated into the protocol, not an additional layer.
- With QUIC, multiple UDP streams are introduced with an embedded TCP model built inside.

Protocol	Key Feature	Problem Solved	New Limit Introduced
HTTP/1.0	Non-persistent connections	-	High overhead, inefficiency
HTTP/1.1	Persistent connections	Connection overhead	HoL Blocking (Application)
HTTP/2	Multiplexing over TCP	HoL Blocking (Application)	HoL Blocking (Transport/TCP)
HTTP/3	Multiplexing over QUIC/UDP	HoL Blocking (Transport/TCP)	-

Tabella 1: Performance evolution of the HTTP protocol.

45 Difference Between HTTP and HTTPS

The primary difference between **HTTP** and **HTTPS** lies in security and data encryption.

- **HTTP** (*Hypertext Transfer Protocol*):
 - Non-secure: data is transmitted in cleartext and is vulnerable to interception.
 - Uses port **80**.
- **HTTPS** (*Hypertext Transfer Protocol Secure*):
 - Secure: uses **SSL/TLS** encryption to protect data.
 - Uses port **443**.
 - Requires a security certificate to establish an encrypted connection.

In summary, while HTTP provides unsecured data transfer, HTTPS guarantees data confidentiality and integrity through encryption.